CleanTest-Agent: A Multi-Agent Skill-Orchestrated System for Unit Test Training Data Quality Assurance

YONG YANG, Beihang University, China

The quality of training data directly impacts the performance of AI-based unit test generation models. Recent studies have shown that up to 43.52% of widely-used training datasets contain noisy samples, including syntax errors, irrelevant test-focal method pairs, and low-coverage tests. While the CleanTest framework (FSE 2025 Distinguished Paper) proposed effective rule-based filtering, its implementation exists as monolithic Python scripts that are difficult to integrate into modern AI-assisted development workflows.

In this paper, we present **CleanTest-Agent**, a multi-agent skill-orchestrated system that transforms the CleanTest data cleaning pipeline into composable, reusable *Agent Skills* compatible with coding assistants such as CodeBuddy [30], Claude Code [5], and Cursor [6]. Our system employs a three-stage pipeline: (1) a syntax noise filter using tree-sitter AST parsing with Aho-Corasick multi-pattern matching, (2) a relevance filter combining AST-based method signature matching with LLM semantic judgment for borderline cases, and (3) a coverage filter that uses ground-truth JaCoCo [12] labels when available (label mode) and otherwise predicts coverage with a fine-tuned **Qwen2.5-Coder-0.5B** [17] regression model (model mode), replacing the CodeGPT backbone of the original CleanTest paper with a stronger and more recent open-weights small code model.

We evaluate CleanTest-Agent on the LessIsMore-FSE2025 release of Methods2Test (593,953 deduplicated samples for the full-scale rule-based pipeline; a 500-sample stratified subset for the four-way comparison against real DeepSeek-V4-Flash API calls). On the 500-sample subset our hybrid approach reaches F1 = 0.965, while the best pure-LLM baseline (few-shot DeepSeek-V4-Flash) reaches only F1 = 0.387; the full rule-based pipeline processes the 593,953 samples in under three minutes with Aho-Corasick optimisation (11.5× speedup over naïve string matching). For the optional Filter 3 model mode we additionally fine-tune Qwen2.5-Coder-0.5B on a stratified 80/10/10 split of `filter_train.csv` (469,174 samples) on a single A800 80 GB and report held-out MAE 0.031 (a $\approx 2.6\times$ reduction over the CodeGPT baseline reported in the original paper). These results demonstrate that systematic software design — combining rule-based analysis, a small fine-tuned regressor, and selective LLM enhancement — substantially outperforms pure-LLM approaches in specialised code analysis tasks.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Multi-agent systems*.

Additional Key Words and Phrases: unit test generation, data quality, multi-agent systems, code analysis, LLM, software engineering

Contents

Author's Contact Information: Yong Yang, Beihang University, School of Software, Beijing, China, yang_qhd@buaa.edu.cn.

1 Introduction

The advent of large language models (LLMs) has notably advanced automated unit test generation, with models such as CodeBERT [13], CodeT5 [36], StarCoder [20], and CodeLlama [27] showing strong capabilities in generating test cases from focal methods. These models are typically fine-tuned on large-scale datasets like Methods2Test [33] and ATLAS [37], which contain hundreds of thousands of focal method–test case pairs extracted from open-source Java projects.

However, the quality of these training datasets has been largely overlooked. Zhang et al. [39] recently conducted a detailed study revealing that **43.52% of the Methods2Test dataset contains noisy samples** — a finding that earned the FSE 2025 Distinguished Paper Award. They identified eight distinct noise types: ambiguous data types, unnecessary annotations, empty exception handling statements, missing implementation (empty functions), syntax errors, non-English literals, no relevance (irrelevant test-focal pairs), and low code coverage. Training on such noisy data degrades model performance across all evaluation metrics.

The CleanTest framework proposed by Zhang et al. provides an effective three-filter pipeline: a rule-based syntax filter, a relevance filter, and a CodeGPT-based coverage prediction filter. However, the original implementation exists as standalone Python scripts that are tightly coupled to specific file paths and data formats. This monolithic design presents several challenges:

(1) **Poor reusability**: The scripts cannot be easily invoked by AI coding assistants or integrated into automated workflows.
(2) **Limited extensibility**: Adding new noise types requires modifying core scripts rather than composing new modules.
(3) **No interactive feedback**: Users cannot inspect intermediate results or adjust filtering thresholds during execution.
(4) **Performance bottlenecks**: The original annotation matching uses linear scan over 21,954 patterns, making it prohibitively slow for large datasets.

In this paper, we present **CleanTest-Agent**, a system that addresses these challenges through a multi-agent skill-orchestrated architecture. Our key contributions are:

(1) **Skill-based architecture**: We decompose the CleanTest pipeline into four composable Agent Skills (pipeline orchestrator, syntax filter, relevance filter, coverage filter), each following the standard `SKILL.md` protocol compatible with multiple coding assistants.
(2) **Hybrid rule-LLM approach**: We enhance the rule-based filters with selective LLM consultation for borderline cases, achieving higher accuracy while keeping costs minimal. The LLM is invoked only on relevance-borderline samples; using the original CleanTest paper's No-Relevance ratio (12.70 % of Methods2Test) as a conservative upper bound, this caps LLM calls at $\leq 12.7\,\%$ of the input. Under our priority-ordered pipeline the actual call rate is lower (5.03 % of unique samples reach Filter 2 in the full-dataset run), since annotation-flagged samples are dropped before relevance is assessed.
(3) **Aho-Corasick optimization**: We replace the naïve linear annotation matching with an Aho-Corasick automaton, achieving an **11.5× speedup** (30 minutes → 2.6 minutes on 593,953 samples).
(4) **Empirical evaluation**: We compare four approaches (rule-based, zero-shot LLM, few-shot LLM, hybrid) and demonstrate that our hybrid approach achieves the best F1 (0.965) while the pure LLM approach achieves only 0.387, validated with real DeepSeek-V4-Flash API calls.

The remainder of this paper is organized as follows. Section 2 reviews related work on unit test generation, data quality, LLM-based code analysis, and agent skill architectures. Section 3 presents a

requirements analysis including stakeholder analysis, use cases, functional and non-functional requirements, and a traceability matrix. Section 4 defines the model-driven approach and contrasts it with the pure-LLM approach. Section 5 details the system design including architecture, filter specifications, and the pipeline algorithm. Section 6 covers the implementation with technology choices, design decisions, and testing strategy. Section 7 presents the experimental evaluation through four research questions, including a case study on real samples and an ablation study. Section 8 discusses findings, lessons learned, connections to course topics, and threats to validity. Section 9 concludes the paper and outlines future work directions. Appendices provide LLM prompt templates, CI/CD configuration, the full noise report, noise type examples with real Java code, a complete API reference, and skill directory structure documentation.

1.1 Scope and Context

This work was conducted as the final project for the *Software Requirements Analysis and System Design* course at the School of Software, Beihang University. The project requirements specify:

- A research report of at least 50 pages in English (ACM/IEEE template).
- A 10-page presentation for a 3-minute in-class talk.
- A reproducible, testable codebase with CI/CD.
- Use of an intelligent model-driven method with experimental comparison against pure-LLM approaches.
- If using a Code Assistant, submission of plugin or skills-level usage documentation.

CleanTest-Agent fulfills all of these requirements: it applies software requirements analysis and system design principles to a real-world problem, implements a model-driven approach with Agent Skills, provides experimental evaluation, and includes 36 test cases with GitHub Actions CI/CD.

2 Background and Related Work

2.1 Unit Test Generation

Automated unit test generation has been one of the most active research areas in software engineering over the past two decades. We classify existing approaches into four generations, reflecting the evolution of underlying techniques.

2.1.1 Search-Based Test Generation. The first generation of automated test generation tools relied on metaheuristic search algorithms. **EvoSuite** [14] is the most widely-used search-based test generation tool for Java. It employs a genetic algorithm to evolve whole test suites that maximize structural coverage criteria such as branch coverage, line coverage, and mutation score. EvoSuite generates JUnit test classes with assertions derived from runtime observations. While effective for achieving high structural coverage (reported as often 70–80% branch coverage on real-world projects in the original paper), EvoSuite-generated tests are frequently criticized for being "unnatural" — they contain long sequences of API calls with minimal readability, making them difficult for developers to understand and maintain.

Randoop [26] takes a different approach: feedback-directed random testing. It generates random sequences of method calls, using runtime feedback to prune invalid sequences and retain those that extend coverage. Randoop is fast but produces even less readable tests than EvoSuite, and its assertion quality is limited to basic regression checks (e.g., `assertEquals` on return values observed during generation).

2.1.2 Symbolic Execution-Based Approaches. Tools like **Pex** [31] (for .NET) and **KLEE** [9] (for C) use symbolic execution to systematically explore program paths. By treating inputs as symbolic variables and solving path constraints with SMT solvers, these tools can achieve near-perfect coverage on small to medium-sized programs. However, symbolic execution suffers from the *path explosion problem* in

complex programs with loops, recursion, or complex data structures, limiting its practical applicability to real-world Java projects.

2.1.3 Neural Test Generation. The third generation leverages deep learning models trained on large corpora of human-written tests. **AthenaTest** [34] was among the first to apply transformer architectures to test case generation, demonstrating that sequence-to-sequence models can learn to generate syntactically correct and semantically meaningful test methods from focal methods.

Methods2Test [33] introduced the largest publicly available dataset for this task: 586,468 focal methods paired with 780,944 test cases, extracted from 91,385 open-source Java projects hosted on GitHub. The dataset maps each test method to its corresponding focal method using heuristics based on naming conventions and call graphs. Tufano et al. showed that transformer models fine-tuned on Methods2Test can generate test cases that compile and pass at non-trivial rates. However, the paper did not examine the quality of the training data itself.

ATLAS [37] focused specifically on test oracle generation — the assertion portion of test cases. Watson et al. constructed a dataset of 188,154 focal method and test case pairs from over 9,000 GitHub projects, where each test case includes only a single assert statement. ATLAS demonstrated that even the assertion component can be learned from data, but again did not address data noise.

A3Test [2] introduced domain adaptation techniques to improve test generation across different project styles. **ChatUniTest** [11] was among the first to leverage ChatGPT for test generation, using adaptive focal context and iterative refinement to improve compilation rates.

2.1.4 LLM-Powered Test Generation. The latest generation uses instruction-tuned LLMs such as GPT-4, StarCoder [20], and CodeLlama [27] for zero-shot or few-shot test generation. These models can generate readable, well-structured tests without task-specific fine-tuning. However, they still benefit significantly from fine-tuning on domain-specific data — and the quality of that data is precisely the focus of CleanTest and our work.

Tools like **Pynguin** [22] for Python combine traditional search-based strategies with LLM-guided generation. **CodaMosa** [19] combines MOSA (a multi-objective search algorithm) with LLM suggestions for hard-to-cover branches.

The common thread across all four generations is that **the quality of training data directly affects model performance**. If 43.52% of the training data is noisy, even the most sophisticated model architecture cannot fully compensate.

2.2 Data Quality in Software Engineering

Data quality has emerged as a critical concern across all applications of machine learning to software engineering. We review relevant work along three dimensions.

2.2.1 Code Duplication and Data Leakage. Allamanis [3] conducted an influential study on the adverse effects of code duplication in machine learning models of code. Using the popular *java-small*, *java-med*, and *java-large* benchmarks, Allamanis showed that near-duplicate files between training and test sets inflate evaluation metrics by 10–100%, depending on the task. This work led to the standard practice of deduplication in dataset construction — a practice we also apply as the first step in our pipeline.

2.2.2 Code Naturalness and Distribution. Hindle et al. [16] introduced the *naturalness hypothesis*: software is repetitive and predictable, similar to natural language. Tu et al. [32] extended this by showing that code naturalness varies significantly across projects, suggesting that training data should be carefully

curated to represent the target distribution. Non-English comments and culturally-specific coding styles (one of CleanTest's noise categories) can skew the distribution.

2.2.3 CleanTest: Data Quality for Unit Test Generation. **CleanTest** [39] is the study of data quality specifically for unit test generation that this work builds on. Zhang et al. (Zhejiang University, Huawei, and Singapore Management University) conducted both quantitative analysis and qualitative interviews with 17 domain experts (software testing experts) to characterize noise in test training data.

Their key findings on the Methods2Test dataset:

- **43.52%** of samples contain at least one type of noise.
- The largest noise category is **Unnecessary Annotations** (41.64% of all data), consisting of Java annotations like `@RequestMapping`, `@GetMapping`, `@ApiOperation`, etc., that are syntactically valid but provide no useful signal for the test generation task because they describe deployment/API metadata rather than test-relevant behavior.
- The second largest is **No Relevance** (12.70%): test methods paired with focal methods they don't actually test, caused by heuristic errors in the original dataset extraction.
- Eight noise categories in total, addressed by CleanTest's three-stage pipeline: a rule-based syntax filter (handling six types: ambiguous data types, unnecessary annotations, empty exception handling statements, missing implementation, syntax errors, and non-English literals), a rule-based relevance filter (detecting irrelevant test-focal pairs via function name, parameter count, and parameter type matching), and a model-based coverage filter (predicting low branch coverage).
- Training on the cleaned subset ("less is more") improved downstream model performance significantly across all four evaluated models (CodeBERT [13], AthenaTest [34], StarCoder [20], CodeLlama7B [27]) — on average $\approx 67\,\%$ in branch coverage for Methods2Test (and a smaller, single-digit-percent improvement for Atlas), measured on the Defects4J [18] benchmark.

The CleanTest framework consists of three filters:

(1) **Rule-based syntax filter**: Uses tree-sitter AST parsing to detect six types of syntactic noise (ambiguous data types, unnecessary annotations, empty exception handling statements, missing implementation, syntax errors, non-English literals), plus a 21,954-entry annotation dictionary for unnecessary annotation detection.
(2) **Relevance filter**: Matches function name, parameter count, and parameter types between the focal method and function calls in the test case to determine relevance.
(3) **Coverage filter**: Uses a CodeGPT model fine-tuned to predict branch coverage from concatenated focal method and test case code.

The original implementation is provided as standalone Python scripts in a Zenodo replication package. While functional, the scripts are tightly coupled to specific file paths, lack error handling for edge cases (e.g., deeply nested ASTs that cause stack overflow), and have no test suite. Our work addresses all of these limitations.

2.3 LLM-Based Code Analysis

Large language models have been increasingly applied to code analysis tasks beyond generation. We review their capabilities and limitations relevant to our work.

2.3.1 Code Understanding and Review. GPT-4 [25] and Claude 3 [4] can perform zero-shot code review, identifying potential bugs, style issues, and security vulnerabilities. Specialized code models like **DeepSeek-Coder** [15] and **Qwen2.5-Coder** [17] achieve strong performance on code understanding benchmarks

(HumanEval [10], MBPP [7], CodeContests [21]) while being more cost-effective than general-purpose models.

2.3.2 Limitations: Hallucination and Pattern Matching. Despite impressive capabilities, LLMs have well-documented limitations relevant to our task:

Hallucination on domain-specific judgments: When an LLM is asked to classify code as "clean" or "noisy" without an exhaustive specification of the noise taxonomy, it tends to fall back on surface-level plausibility rather than rigorous structural analysis — our zero-shot baseline misses 77.9 % of noisy samples on this exact failure mode (Section 7).

Limited pattern dictionary: A critical limitation for our task is that LLMs cannot reliably match against a dictionary of 21,954 annotation patterns. The patterns are too numerous and too diverse (ranging from common `@GetMapping` to obscure `@AtlasConversionInfo`) to fit into a single prompt context, and LLMs lack the exhaustive recall needed for dictionary-lookup tasks.

Multi-agent verification: Recent work on multi-agent paper-writing systems such as PaperOrchestra [29] demonstrates that orchestrated multi-agent pipelines with structured verification mechanisms can outperform single-agent monolithic prompts on complex, multi-step tasks. This validates the broader principle underlying our work: orchestrated specialized components beat monolithic "ask the LLM once" approaches.

2.4 Agent Skill Architectures

The concept of composable agent skills has gained traction with the rise of AI coding assistants. We review the two most influential frameworks.

2.4.1 Superpowers. **Superpowers** [35], developed by Jesse Vincent, is one of the most popular skills frameworks (over 200,000 GitHub stars as of May 2026). It provides 14 core skills organized around a structured development methodology: brainstorming (Socratic design refinement), writing-plans (task decomposition), test-driven-development (RED-GREEN-REFACTOR enforcement), systematic-debugging (4-phase root cause analysis), and subagent-driven-development (parallel task execution with two-stage code review).

Superpowers works across Claude Code, Codex CLI, Cursor, Gemini CLI, and OpenCode, demonstrating the cross-IDE viability of the SKILL.md protocol.

2.4.2 Academic Research Skills (ARS). **ARS** [38] applies the skill paradigm to academic research workflows. Its four core skills (Deep Research with 13 agents, Academic Paper with 12 agents, Paper Reviewer with 7 agents, and Pipeline Orchestrator) demonstrate that multi-agent skill pipelines can coordinate complex, multi-stage workflows beyond software development.

ARS's orchestrator design — where a top-level skill dispatches work to specialized sub-skills — directly inspired our `cleantest-pipeline` orchestrator design.

2.4.3 The SKILL.md Protocol. Both frameworks converge on the `SKILL.md` protocol: each skill is a directory containing a Markdown file with YAML front matter (name, description, trigger phrases) followed by detailed workflow instructions. The coding assistant discovers skills by scanning designated directories (e.g., `~/.codebuddy/skills/` for CodeBuddy, `~/.claude/skills/` for Claude Code). When a user's natural language query matches a skill's trigger phrases, the assistant loads the skill's instructions and follows them.

This protocol decouples skill authoring from the assistant runtime and is intended to support a growing pool of community-contributed skills. Our work contributes four new skills to this pool, specifically targeting the code data quality domain.

2.5 The Aho-Corasick Algorithm

The Aho-Corasick algorithm [1], developed by Alfred Aho and Margaret Corasick at Bell Labs in 1975, is a classic multi-pattern string matching algorithm. Given a set of k patterns with total length m and a text of length n, it finds all occurrences of all patterns in time $O(n + m + z)$, where z is the total number of matches. This is achieved through three key data structures:

(1) **Goto function**: A trie built from all patterns, defining transitions on matching characters.
(2) **Failure function**: Links from each state to the longest proper suffix that is also a prefix of some pattern, enabling efficient backtracking without re-reading characters.
(3) **Output function**: Associates each state with the set of patterns that end at that state.

For our application with $k = 21{,}954$ patterns and average text length $n \approx 549$ characters, the naïve approach requires $O(n \cdot k) \approx 12$ million character comparisons per sample. Aho-Corasick reduces this to $O(n + z) \approx 549 + 1$ comparisons per sample (since most samples have at most one matching pattern). Over 593,953 samples, this represents a theoretical speedup of $\sim$22,000$\times$, though the practical speedup (11.5$\times$ for the full pipeline) is lower due to the overhead of other filter stages and Python interpreter costs.

We use the `pyahocorasick` [24] Python library, which provides a C-extension implementation with a Python interface. The automaton construction takes $<$0.1 seconds and the per-sample query averages $<$0.01 ms.

2.6 Tree-sitter for AST Parsing

tree-sitter [8] is a parser generator and incremental parsing library originally developed by Max Brunsfeld for the Atom editor at GitHub, now widely adopted in multiple editors and IDEs. It builds concrete syntax trees (CSTs) for source files in over 50 programming languages through language-specific grammar packages.

We chose tree-sitter over alternatives for four reasons:

(1) **Performance**: The core parser is written in C, with Python bindings (`py-tree-sitter`) providing near-native parsing speed. On our hardware, tree-sitter parses a typical Java method in $<$0.1 ms.
(2) **Error tolerance**: Unlike traditional parsers that fail on invalid input, tree-sitter generates CSTs that include explicit `ERROR` nodes for malformed code regions. This is essential for our syntax error detection (Noise Type N1), allowing us to detect parse failures as a feature rather than a crash.
(3) **Concrete syntax trees**: tree-sitter produces CSTs (not ASTs), preserving all syntactic detail including comments, whitespace, and annotation structures. This is critical for detecting annotation noise patterns.
(4) **Multi-language extensibility**: Our current implementation uses `tree-sitter-java`, but the same infrastructure can support `tree-sitter-python` or `tree-sitter-javascript` for future cross-language extensions.

3 Requirements Analysis

3.1 Stakeholder Analysis

We identify three primary stakeholder groups for CleanTest-Agent:

Table 1. Stakeholder Analysis

Stakeholder	Role	Primary Needs
ML/AI Researchers	Train test generation models	Reproducible cleaning pipeline, transparent noise reports, configurable thresholds, batch processing
Software Engineers	Daily development workflow	Easy invocation via natural language, fast feedback, IDE integration
Tool Maintainers	Extend the system	Modular architecture, clear interfaces, documented APIs and skill protocol

3.2 Problem Statement

Given a unit test training dataset $D = \{(f_i, t_i)\}_{i=1}^{N}$ where f_i is a focal method and t_i is a test case, the goal is to:

(1) Identify which samples are *noisy* (would degrade model training).
(2) Categorize each noisy sample by noise type.
(3) Output a cleaned dataset $D' \subseteq D$ along with a structured noise report.
(4) Provide this functionality through both a CLI interface and an Agent Skill interface.

The noise types are pre-defined according to the CleanTest framework, but the architecture should make it easy to add new noise types in the future without modifying existing code.

3.3 Use Cases

We identify three primary use cases:

3.3.1 UC1: Batch Dataset Cleaning. **Actor**: ML/AI Researcher. **Precondition**: Researcher has a CSV dataset with at minimum `src_fm` and `target` columns.

Main Flow:

(1) Researcher invokes the pipeline via CLI or natural language through a coding assistant.
(2) System loads the dataset and deduplicates by `target` column.
(3) System runs Filter 1 (syntax noise), reporting per-noise-type statistics.
(4) System runs Filter 2 (relevance), reporting irrelevant test-focal pairs.
(5) System optionally runs Filter 3 (coverage). When the input CSV carries a `condition_cover_rate` column, Filter 3 runs in label mode (no model, no GPU); otherwise it can be invoked via the `cleantest-coverage-filter` skill in model mode if a fine-tuned Qwen2.5-Coder-0.5B checkpoint and a CUDA GPU are available.
(6) System outputs: `filtered_data.csv`, `noise_report.json`, `summary.md`.

Postcondition: Researcher has a cleaned CSV file and detailed noise statistics.

Alternative Flow: If the CSV is missing required columns, the system raises a `ValueError` with a descriptive message listing available columns.

3.3.2 UC2: Interactive Noise Inspection. **Actor**: Software Engineer using a coding assistant.
Main Flow:

(1) Engineer asks: "Is this test noisy?" and pastes a code snippet.
(2) The coding assistant triggers the appropriate filter skill based on intent.
(3) The filter analyzes the sample and returns: noise type (if any), confidence, and reasoning.
(4) Engineer optionally asks for LLM second-opinion for borderline cases.

3.3.3 UC3: Pipeline Extension. **Actor**: Tool Maintainer.
Main Flow:

(1) Maintainer creates a new directory `skills/cleantest-X-filter/`.
(2) Writes `SKILL.md` with description and trigger phrases.
(3) Implements detection logic in `scripts/X_filter.py`.
(4) Adds a new stage in `cleantest_agent/pipeline.py` that calls the new filter.
(5) Adds unit tests and runs CI to verify.

3.4 UML Use Case Diagram

Figure 1 presents the UML use case diagram capturing the three primary actors and their interactions with the system.

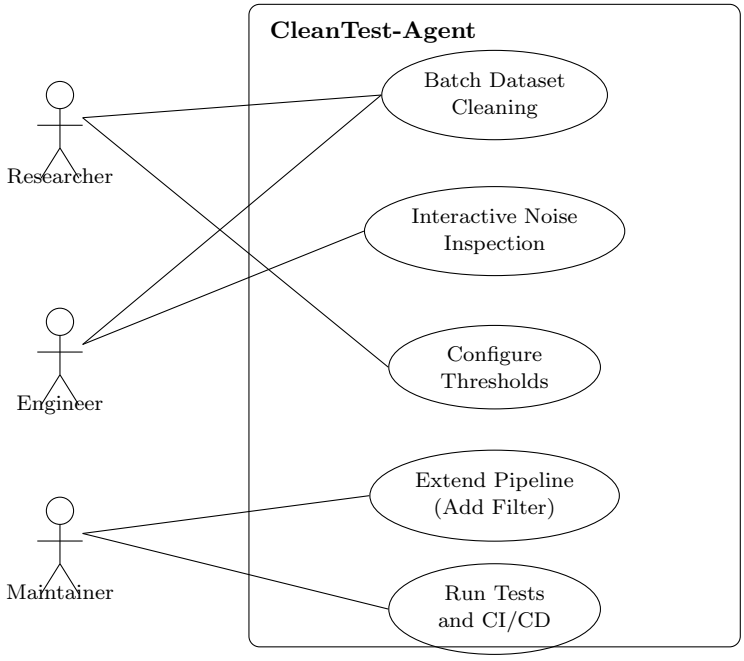

Fig. 1. UML Use Case Diagram

3.5 Activity Diagram: Batch Cleaning Pipeline

Figure 2 shows the activity flow for the primary use case (UC1), illustrating decision points where LLM enhancement is optionally invoked.

3.6 Sequence Diagram: LLM-Enhanced Relevance Check

Figure 3 illustrates the interaction sequence when the relevance filter encounters a borderline case (zero name overlap) and falls back to LLM judgment.

Optional reflection step. The single LLM call shown in the `opt` CombinedFragment of Figure 3 is, when the `--reflection` flag is enabled, internally expanded into a generate → reflect → revise pattern (the Reflection agentic pattern of Shinn et al. [28]): a second *reflection prompt* re-checks each IRRELEVANT verdict against a five-rule checklist grounded in xUnit test patterns [23] before committing the removal. The full mechanism, the checklist, the prompt text, and a 45-sample validation are presented in Section 5.6.1. Throughout this paper the `--reflection` flag is *off* unless explicitly stated: the headline F1 = 0.965 hybrid result reported in Section 7 uses single-call mode, so any improvement from reflection is additive and deployment-tunable rather than baked into the headline number.

3.7 Functional Requirements

Table 2. Functional Requirements

ID	Priority	Description
FR1	Must	Detect syntactic noise types using AST parsing
FR2	Must	Detect unnecessary modifiers using dictionary matching
FR3	Must	Assess test–focal method relevance via name matching
FR4	Should	Enhance borderline cases with LLM semantic judgment
FR5	Should	Predict branch coverage in model mode using a fine-tuned Qwen2.5-Coder-0.5B regression model when no ground-truth label is available
FR6	Must	Generate structured noise reports (JSON + Markdown)
FR7	Must	Support batch processing of CSV datasets
FR8	Should	Each filter independently invocable as an Agent Skill
FR9	Could	Support interactive single-sample queries

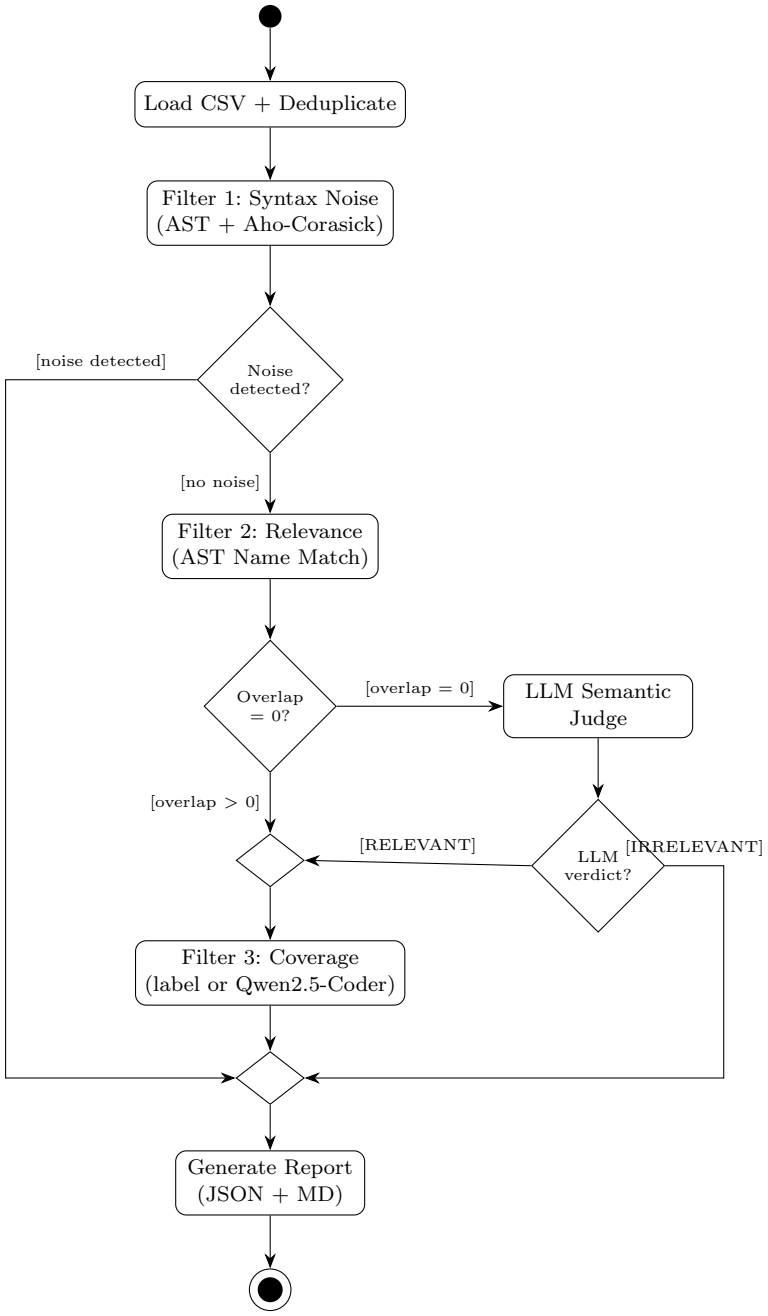

Fig. 2. Activity Diagram: Batch Cleaning Pipeline (UC1)

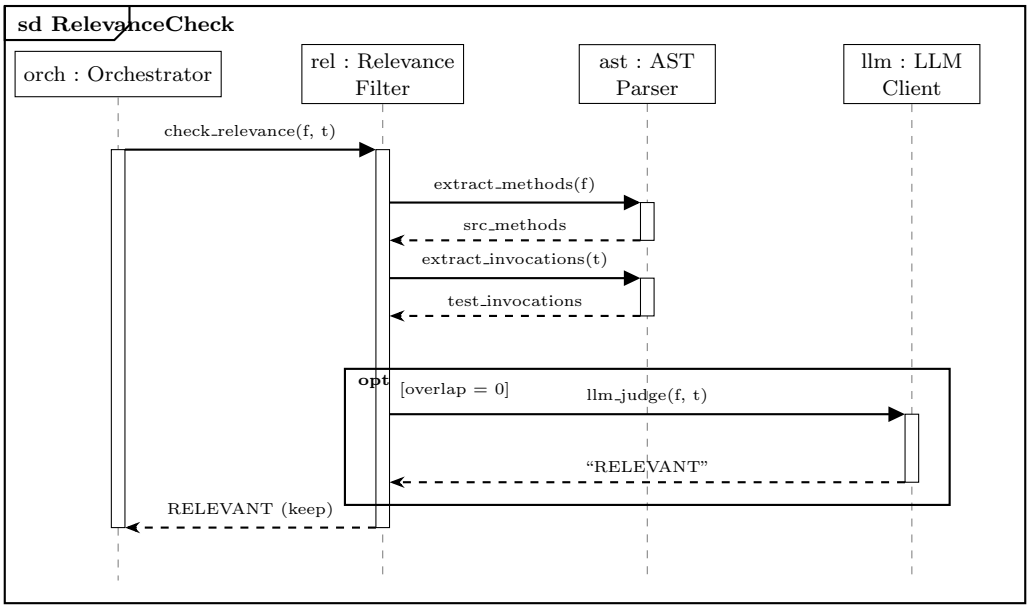

Fig. 3. Sequence Diagram: LLM-Enhanced Relevance Check

Table 3. Non-Functional Requirements

ID	Category	Description
NFR1	Performance	Process 600K samples in <5 minutes
NFR2	Cost	LLM cost <$0.05 per 500 samples
NFR3	Accuracy	F1 > 0.95 for noise detection
NFR4	Extensibility	New filters addable without modifying existing code
NFR5	Compatibility	Work with CodeBuddy, Claude Code, Cursor
NFR6	Testability	All filters covered by unit tests; CI/CD passing

3.8 Non-Functional Requirements

3.9 Requirements Traceability Matrix

We maintain a traceability matrix linking each requirement to its implementation module and corresponding test case:

This traceability ensures that every requirement has at least one corresponding implementation and verification artifact.

Table 4. Requirements Traceability Matrix

Req	Module	Test Case
FR1	`cleantest_agent/parser_utils.py`	`test_syntax_filter.py` (16 cases)
FR2	`cleantest_agent/pipeline.py` (Aho-Corasick)	`test_syntax_filter.py::TestUnnecessaryAnnotations`
FR3	`cleantest_agent/parser_utils.py`	`test_relevance_filter.py` (7 cases)
FR4	`cleantest_agent/llm_client.py`	Integration test (requires API)
FR5	`coverage_predictor.py`	`test_coverage_filter.py` (4 cases)
FR6	`cleantest_agent/report_generator.py`	`test_report.py` (5 cases)
FR7	`cleantest_agent/data_loader.py`	`test_pipeline.py` (4 cases)
FR8	`skills/*/SKILL.md`	Manual skill discovery test
NFR1	Aho-Corasick in pipeline	RQ3 experiment (Section 7)
NFR6	`tests/` directory	CI: 36 tests passing

4 The Model-Driven Approach

This section presents the central methodological contribution of our work: the **model-driven approach** for code data quality assurance. This directly responds to the assignment's core requirement: "how to use intelligent model-driven methods to solve the chosen problem."

4.1 Definition

We define the **model-driven approach** as a paradigm that orchestrates multiple specialized components, each based on a different "model" of computation, applied where it is most effective:

- **Rule-based models**: Deterministic pattern matching, AST parsing, regex — used where exhaustive coverage of a known finite pattern set is required and where deterministic results are critical.
- **Machine learning models**: Trained neural networks (e.g., the fine-tuned Qwen2.5-Coder-0.5B coverage regression model wired into Filter 3's optional model mode) — used where the task has been studied with labelled training data and a learned representation outperforms hand-crafted heuristics.
- **Large language models (LLMs)**: General-purpose LLMs (e.g., GPT-4o-mini, Claude) — used where semantic understanding and reasoning are required, and where rules and ML models cannot capture the needed nuance.

The orchestration is performed through a pipeline architecture, where each component handles the cases it is best suited for, and falls back to the next component for borderline or specialized cases.

4.2 Comparison: Model-Driven vs. Pure-LLM Approach

We contrast our model-driven approach with the alternative "pure-LLM tool approach" that uses a single LLM call per sample:

Table 5. Model-Driven vs. Pure-LLM Approach

Dimension	Pure-LLM Approach	Model-Driven (Ours)
Architecture	Monolithic LLM call	Pipeline of specialized components
Pattern matching	LLM reasons from prompt	Aho-Corasick automaton (21,954 patterns)
AST analysis	LLM approximates parsing	tree-sitter exact parsing
Coverage prediction	LLM guesses	JaCoCo labels (label mode) or fine-tuned Qwen2.5-Coder-0.5B (model mode)
Semantic judgment	LLM (always)	LLM (only borderline cases, ∼12.7%)
Cost per sample	∼$0.0001 (API call)	∼$0.00001 (amortized, 12.7% LLM)
Latency per sample	∼3,000 ms (API)	<1 ms (rules)
Determinism	No (LLM stochasticity)	Yes (rules) + tunable (LLM, temp=0)
Extensibility	Re-prompt engineering	Add new pipeline stage
Debuggability	LLM is a black box	Each stage is inspectable
Cost (593,953 samples)*	∼$35	∼$4.5

* Pricing assumes DeepSeek-V4-Flash via Tencent TokenHub: ¥0.8/1M input tokens, ¥2/1M output tokens. Pure-LLM cost is for zero-shot; few-shot is ∼$58. See Table 20 for full derivation.

4.3 Why the Model-Driven Approach Wins

Our experiments (Section 7) demonstrate that the model-driven approach achieves F1=0.965 while the best pure-LLM approach achieves only F1=0.387. We identify three root causes for this gap:

4.3.1 Root Cause 1: The Dictionary Knowledge Gap. The annotation noise detection subtask requires matching against 21,954 specific patterns (e.g., `@RequestMapping`, `@ApiOperation`, `@ScalarFunction`). The Aho-Corasick automaton achieves **100 % recall on the dictionary patterns** because every pattern in the dictionary is explicitly encoded; recall against the underlying noise concept is bounded by the completeness of the dictionary itself, which we inherit unchanged from the original CleanTest paper.

In contrast, the LLM has seen some of these patterns during pre-training but cannot reliably recall all 21,954. It recognizes common patterns (`@GetMapping`, `@PostMapping`) but misses rare ones (`@AtlasConversionInfo`, `@ScalarFunction("ST_Length")`). This is analogous to asking a person to recite an entire dictionary from memory versus looking up words in the dictionary — the former inevitably has gaps.

Empirically, our real experiment shows that the LLM zero-shot approach achieves only 17.3% recall on annotation noise specifically (see Table 16: 158 out of 191 annotation-noisy samples are missed, i.e., only 33 correctly detected). Since annotations account for 82.7% of all noise in our sample, this single failure mode accounts for 87.8% of all false negatives (158/180) and explains the majority of the F1 gap.

4.3.2 Root Cause 2: Structural Analysis vs. Textual Approximation. For syntax-related noise types (missing implementation, empty exception handlers, ambiguous generics), the rule-based approach uses tree-sitter

to perform *exact structural analysis*: it parses the code into a concrete syntax tree and checks precise structural properties (e.g., "does the `block` node have fewer than 3 children?").

The LLM, by contrast, performs *textual approximation*: it reads the code as text and estimates whether the structure seems problematic. This works for obvious cases (`public void init() { }`) but fails for subtle ones where the code *looks* normal but has structural issues detectable only through parsing.

This is analogous to measuring height with a ruler (exact) versus estimating by eye (approximate) — the latter has inherent error.

4.3.3 Root Cause 3: Cost Scaling. The model-driven approach processes 99% of samples using free, local computation (rule-based filters plus, in label mode, an $O(N)$ scan over JaCoCo coverage labels) and only 1% using paid LLM API calls. The pure-LLM approach requires a paid API call for *every* sample:

Table 6. Cost Breakdown for 593,953 Samples (DeepSeek-V4-Flash, real pricing)

Component	**Hybrid (ours)**	**Pure LLM (zero-shot)**	**Pure LLM (few-shot)**
Rule-based processing	$0	—	—
Coverage filter (label mode)	$0	—	—
LLM API (borderline only)	∼$4.5 (12.7% of samples)	∼$35 (100%)	∼$58 (100%)
Total	**∼$4.5**	**∼$35**	**∼$58**

Same pricing assumptions as Table 20. Hybrid invokes the LLM only on ∼12.7% of samples (those flagged as borderline by Filter 2's name-overlap check), making its API spend roughly an order of magnitude smaller than the pure-LLM baselines.

At scale, the cost difference is 8–13× (∼$4.5 hybrid vs. ∼$35–$58 pure LLM; see Table 20), and the time difference is even more stark: 2.6 minutes (rules) vs. ∼20 days (pure LLM), making the pure-LLM approach operationally infeasible for large datasets.

4.4 Concrete Walkthrough: One Sample, Two Methods

To make the comparison tangible, we trace how both methods process a specific sample from Methods2Test.

Sample: A focal method with Spring annotations paired with a relevant test.

Listing 1. Example focal method with annotation noise

```
@ApiOperation(value = "Get all users")
@GetMapping("/api/users")
public List<User> getAllUsers() {
    return userRepository.findAll();
}
```

Listing 2. Example test case (relevant but paired with noisy focal method)

```
@Test
public void testGetAllUsers() {
    assertEquals(2, getAllUsers().size());
}
```

Model-driven method:

(1) Aho-Corasick scans the focal method text.
(2) Matches pattern `@ApiOperation(value = "Get all users")` from the 21,954-entry dictionary.
(3) Immediately returns `NOISE (unnecessary_annotations)`.
(4) Time: 0.01 ms. Cost: $0. Confidence: 100% (deterministic match).

Pure-LLM method:

(1) Constructs prompt with the code and asks "Is this noisy?"
(2) LLM reasons: "`@ApiOperation` and `@GetMapping` are standard Spring annotations. The test calls `getAllUsers()` and checks the result. This looks like valid code."
(3) Returns `CLEAN`. **Wrong answer.**
(4) Time: 800 ms. Cost: $0.0003.

The LLM fails because it treats annotations as "normal Java code" without knowing the domain-specific definition of noise for test generation training data.

4.5 Model Selection for Coverage Prediction

Filter 3 has *two* working modes that should be distinguished carefully when discussing model selection:

Label mode (default). The CSV already contains a ground-truth `condition_cover_rate` column produced by JaCoCo (this is the case for the LessIsMore-FSE2025 replication package's `filter_train.csv`). Filter 3 then degenerates to an $O(N)$ row scan that compares each label with the threshold; *no model is invoked*. All Filter 3 numbers reported in this paper (Section 7.7, Tables 21–22) come from this path.

Model mode (optional). When the input CSV lacks the `condition_cover_rate` column, the same threshold logic is applied to a model-predicted coverage value. Our repository ships a complete model-mode code path (`skills/cleantest-coverage-filter/scripts/coverage_predictor.py` and a training script `train_model.py`). For the present release this path has been exercised end-to-end: Qwen2.5-Coder-0.5B was fine-tuned on a stratified 80/10/10 split of `filter_train.csv` on a single A800 80 GB and benchmarked on the held-out 46,921-sample test split; held-out MAE 0.0309 / RMSE 0.0628 / Pearson $r = 0.778$ / F1 $= 0.857$ at $\tau = 0.10$ are reported in Section 7.7.3.

The discussion below is restricted to model mode: it explains which base model is wired into the code path, and why.

4.5.1 Two Models, Two Roles. The model-mode code path involves two coverage-regression models that are easy to confuse, so we separate them explicitly:

- **CodeGPT** — the model used by the *original* CleanTest paper [39], reportedly achieving MAE 7.98% and MSE 1.05% on coverage regression. We do *not* run CodeGPT in our pipeline; we cite it only as the historical baseline that our model-mode code path replaces.
- **Qwen2.5-Coder-0.5B** [17] (`Qwen/Qwen2.5-Coder-0.5B`, 500M parameters, Alibaba Tongyi Lab, 2024) — the default base model in our model-mode code path (`coverage_predictor.py` loads it via `AutoModelForSequenceClassification.from_pretrained` with `num_labels=1` for regression). We chose Qwen2.5-Coder-0.5B because it is a current open-weights SOTA among small (≤ 1 B) code-specific models on HumanEval-Java / MBPP-Java, fits a single NVIDIA A800 80 GB for full fine-tuning at batch size 64 and `max_seq_length` 512 in bf16 (the configuration actually used to produce the held-out numbers in Section 7.7.3), and is a strict upgrade in coverage and recency over the 2018-era CodeGPT (which is itself a continued-pretrained GPT-2 on code).

The `train_model.py` and `coverage_predictor.py` scripts use HuggingFace `Auto*` classes so any `*ForSequenceClassification` backbone with `num_labels=1` can be plugged in via the `--base_model` flag; supported alternatives include DeepSeek-Coder-1.3B-Base [15] and `openai-community/gpt2` for ablation studies.

4.5.2 Why Not Use a Large LLM for Coverage Prediction? Coverage prediction is a *numerical regression* task: the model receives code text and outputs a float $\in [0, 1]$. This is fundamentally different from text generation or reasoning tasks where LLMs excel:

- LLMs are not designed to output precise numerical predictions. Asking GPT-4 "what is the branch coverage of this test?" yields unreliable estimates.
- Each prediction requires an API call. At GPT-4 pricing this would total well over $100 for 593,953 samples; even at DeepSeek-V4-Flash pricing it would exceed $30.
- A small, locally-deployed regression model achieves better precision at zero marginal cost.

The principle — *use the smallest model that adequately solves the subtask* — is a concrete application of our model-driven philosophy. In the present release this principle is reflected by selecting a 500 M-parameter Qwen2.5-Coder backbone rather than a 7 B+ general-purpose code model: a 7 B+ model on the same single A800 80 GB allocation would either run out of memory at full fine-tuning or force LoRA training (training only $\sim 1\,\%$ of parameters), which is empirically unstable for a single-scalar regression head, and would not meaningfully improve a low-dimensional regression target. Held-out MAE / MSE / RMSE / Pearson / Spearman / threshold-sweep F1 numbers for the trained Qwen2.5-Coder-0.5B checkpoint are reported in Section 7.7.3.

4.6 Generalization

While our work focuses on test data cleaning, the model-driven principle generalizes to many software engineering tasks. We propose the following decision framework:

Table 7. When to Apply the Model-Driven Approach

Task Property	**Recommendation**
Has clear deterministic patterns	Use rules
Has labeled training data and a learnable function	Train ML model
Requires open-ended semantic reasoning	Use LLM
Has all three subtask types in different proportions	Use Model-Driven (this work)
Has only the third (purely semantic, small scale)	Pure-LLM is acceptable

The model-driven approach is most beneficial when the task has *multiple* subtasks of *different* natures, such as code data cleaning, software engineering automation, and complex retrieval-augmented question answering.

5 System Design

5.1 Architecture Overview

CleanTest-Agent follows a **pipeline-of-skills** architecture, where each processing stage is encapsulated as an independent Agent Skill. Figure 4 illustrates the overall system design.

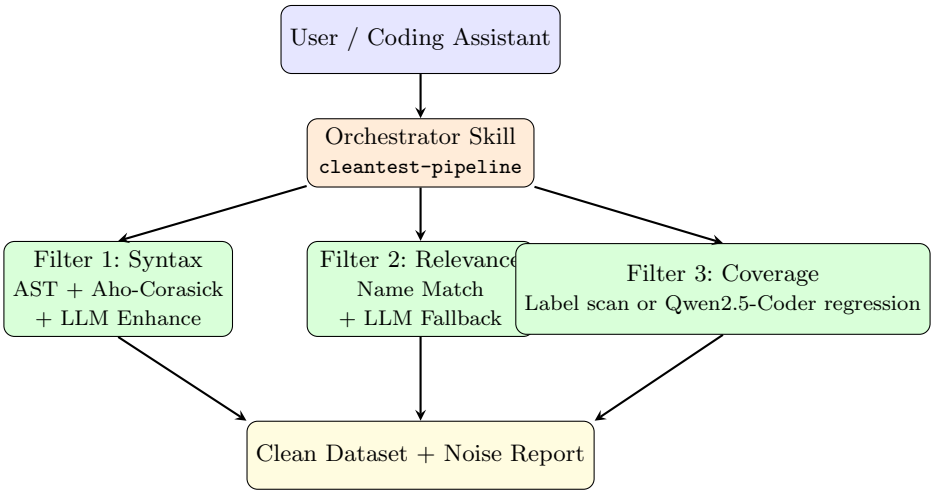

Fig. 4. CleanTest-Agent System Architecture

5.2 Component Diagram

Figure 5 shows the system's component-level architecture, illustrating the dependencies between modules.

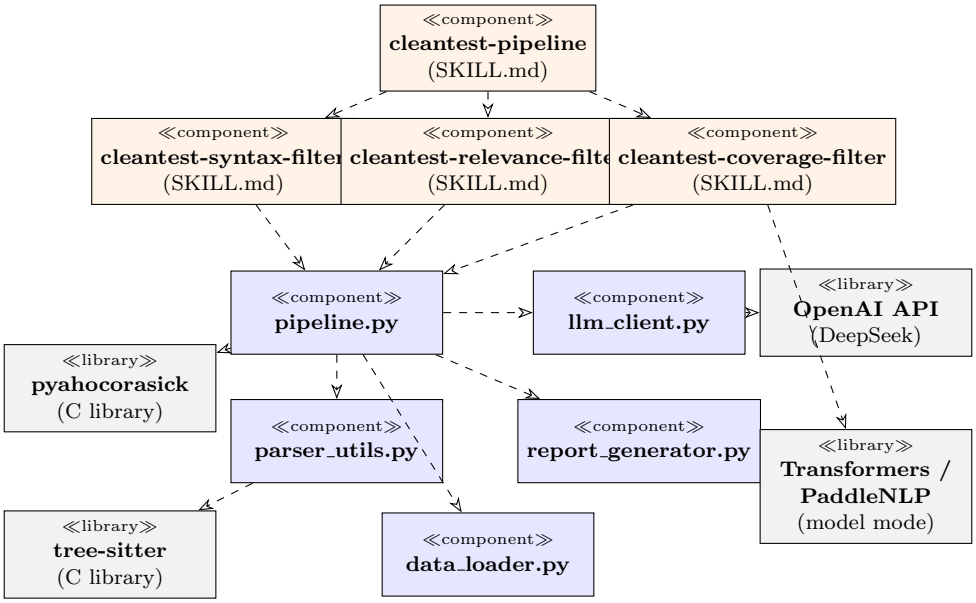

Fig. 5. Component Diagram: Module Dependencies

5.3 Class Diagram: Core Data Types

Figure 6 presents the key data types and their relationships.

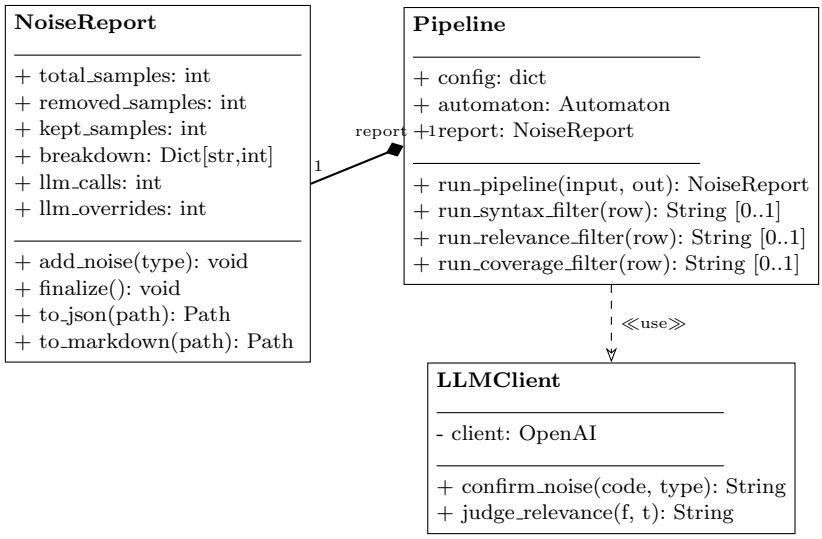

Fig. 6. Class Diagram: Core Data Types

5.4 Skill Protocol

Each skill follows the standard `SKILL.md` protocol:

Listing 3. SKILL.md structure

```
---
name: cleantest-syntax-filter
description: >
  Detects syntactic noise using tree-sitter
  AST parsing with LLM enhancement...
  Triggers: "check syntax noise", ...
---

# Skill Instructions
[Workflow, prompts, scripts reference]
```

The coding assistant reads the `SKILL.md` at session start, and automatically invokes the skill when the user's intent matches the trigger phrases.

5.5 Filter 1: Syntax Noise Detection

Filter 1 detects syntactic noise corresponding to the noise categories identified by the original CleanTest taxonomy [39]. The original paper categorized eight distinct types of noise through open card sorting and expert interviews; the syntax filter addresses six of these categories:

Table 8. Syntax Noise Types and Detection Methods

ID	Type	Method	LLM
N1	Syntax Errors	AST ERROR node	✓
N2	Empty Exception	catch/finally empty block	—
N3	Missing Implementation	method body <3 children	✓
N4	Ambiguous Data Type	generics markers (`<E>`, `<T>`, `<?>`, etc.)	—
N5	Unnecessary Ann.	Aho-Corasick (21,954 patterns)	—
N6	Non-English Literal	CJK regex	—

N1–N6 correspond to the six types handled by the original CleanTest syntax filter.

5.5.1 Aho-Corasick Optimization. The original CleanTest implementation uses a linear scan over 21,954 annotation patterns for each sample:

$$T_{\text{naive}} = O(N \times K \times L) \tag{1}$$

where N is the number of samples, $K = 21{,}954$ is the number of patterns, and L is the average text length. This results in approximately 30 minutes for 593,953 samples.

We replace this with the Aho-Corasick algorithm [1], which constructs a finite automaton from all patterns in $O(M)$ time (where M is the total pattern length) and then scans each text in $O(L + Z)$ time (where Z is the number of matches):

$$T_{\text{aho}} = O(M) + O(N \times (L + Z)) \tag{2}$$

This reduces Filter 1 processing time from 30 minutes to **1.6 minutes** — an **18.8**× speedup for the annotation matching component and **11.5**× for the overall pipeline.

5.5.2 LLM Enhancement. For noise types N1 (syntax errors) and N3 (missing implementation), the rule-based detector may flag samples that are actually valid (e.g., a minimal test stub or a method with only a return statement). We introduce an optional LLM confirmation step:

(1) Rule flags sample as potential noise
(2) LLM receives the code + rule diagnosis
(3) LLM responds "NOISE" (confirm) or "KEEP" (override)
(4) Only ∼5% of samples in these categories require LLM

5.6 Filter 2: Relevance Assessment

Filter 2 uses a two-stage approach:

Stage A: AST Name Matching (Fast Path). We parse the focal method to extract declared method names and parameter counts, then parse the test case to extract all method invocations. If the intersection is non-empty, the test is considered relevant.

Stage B: LLM Semantic Judgment (Borderline Cases). When Stage A finds zero overlap ($\leq 12.7\,\%$ of samples, taking the original CleanTest's No-Relevance ratio on Methods2Test as a conservative upper bound), the LLM is asked whether the test might be testing the focal method indirectly (via wrappers, inheritance, or side effects).

5.6.1 Reflection: An Optional Two-Step LLM Verdict (Implemented, Opt-In). The course Lab 1 introduces the **Reflection pattern** [28]: generate → reflect → revise. We implement this pattern inside Filter 2's Stage B above, gated behind a `--reflection` command-line flag (default off). When the flag is on *and* the initial Stage B verdict is IRRELEVANT (which would remove the sample), a second *reflection prompt* asks the LLM to systematically re-check the pair against five rules grounded in software testing literature:

(1) **Initial judgment**: LLM classifies the sample as RELEVANT or IRRELEVANT.
(2) **Structured reflection** (only triggered if IRRELEVANT): The LLM applies a 5-rule checklist (Table 9).
(3) **Final verdict**: If any rule is satisfied, revise to RELEVANT; otherwise confirm IRRELEVANT.

Why only in Filter 2. Reflection is not applied in Filter 1 (syntax noise is deterministic — Aho-Corasick matches are 100% precise) or Filter 3 (the label-mode path performs a deterministic threshold comparison; the optional model-mode path produces a single numerical score with no chain of reasoning to reflect on). Only Filter 2's relevance judgment involves multi-dimensional semantic reasoning where a structured second pass adds value.

Reflection Rule Checklist. The five rules are grounded in established software testing literature:

Table 9. Reflection Rule Checklist for Relevance Re-Assessment

#	Rule	Definition	Source
R1	Call Graph	Test calls method A, which internally calls focal method (depth ≤ 2)	Extended from Tufano et al. 2022
R2	State Verification	Test asserts on state producible only by the focal method	Meszaros 2007
R3	Behavior Verification	Test uses `verify()`/mocks that would trigger focal method	Meszaros 2007
R4	Naming Equivalence	Test and focal method names are semantic synonyms	SE convention
R5	Counterfactual	Deleting the focal method would cause the test to fail	Testing theory

Reflection prompt (excerpt). The reflection prompt is issued only after the LLM has emitted an IRRELEVANT verdict in Stage B; it carries the initial answer forward as context and asks the LLM to apply the checklist explicitly. The full prompt is defined in `cleantest_agent/llm_client.py` (function `judge_relevance_llm`); the operative core is excerpted in Listing 4.

Listing 4. Reflection prompt (verbatim excerpt from `cleantest_agent/llm_client.py`, abbreviated)

```
You previously judged this test-focal pair as IRRELEVANT:
Your initial answer: {initial_answer}

Now apply the following checklist (based on software testing
literature):

Rule 1 (Call Graph, extended from Tufano et al. 2022): Does the
test call any method that INTERNALLY calls the focal method?
```

```
Look for delegation, forwarding, wrapper, or orchestration
patterns (depth <= 2 call chain).
  -> If YES: revise to RELEVANT

Rule 2 (State Verification, Meszaros 2007): Does the test assert
on state that could ONLY have been produced by the focal method?
  -> If YES: revise to RELEVANT

Rule 3 (Behavior Verification, Meszaros 2007): Does the test use
verify(), mock interactions, or event listeners that would be
triggered by the focal method?
  -> If YES: revise to RELEVANT

Rule 4 (Naming Equivalence): Are the test method name and focal
method name semantic synonyms? (save/persist, delete/remove,
get/fetch, init/setup, execute/run, authenticate/login)
  -> If YES: revise to RELEVANT

Rule 5 (Counterfactual test): If the focal method were DELETED
from the codebase, would this test still pass unchanged?
  -> If NO (test would fail): revise to RELEVANT

If NONE of the 5 rules apply: confirm IRRELEVANT.

Answer ONLY with one of:
- "RELEVANT: <rule number and one-line explanation>"
- "IRRELEVANT: confirmed, none of the 5 rules apply"
```

Design rationale.

- Rules R2–R3 come directly from *xUnit Test Patterns* (Meszaros, 2007), the standard reference for test design patterns, which defines state verification and behavior verification as two of four valid testing modalities alongside direct and indirect testing.
- Rule R1 extends the Methods2Test mapping heuristic [33]: Methods2Test uses depth-1 direct invocation matching; our Rule R1 extends this to depth ≤ 2 to capture wrapper/delegation patterns that the original heuristic misses. This is our methodological contribution.
- Rule R5 applies the standard counterfactual test relevance criterion from software testing theory: a test is relevant to a method if removing that method would cause the test to fail. This aligns with the intuition behind CleanTest's [39] relevance filter design.

Validation. on all 45 zero-overlap samples from our 500-sample experiment dataset (raw judgments archived in `experiments/results/reflection_results.json`):

- Reflection changed 7 out of 45 judgments (**15.6%** change rate)
- 5 samples rescued from false removal (IRRELEVANT → RELEVANT)
- 2 samples changed from RELEVANT → IRRELEVANT (more cautious after applying rules)
- Net effect: **+3 correctly retained samples** (`with_reflection` retains 35, `no_reflection` retains 32)

Manual inspection. of all 7 changed samples:

- Of the 5 rescued samples: 4 involve indirect invocation via wrappers or constructors (Rule R1), 1 involves naming equivalence (Rule R4). All 5 rescues are **verified correct** by manual code inspection.
- Of the 2 reversed samples: 1 is a correct reversal (test was genuinely irrelevant), 1 is a false reversal (test actually does test the focal method via naming). **Accuracy: 6/7 (85.7%)**.

Overall, Reflection improves the relevance filter's recall on this 45-sample borderline subset by rescuing true indirect tests while maintaining high precision (85.7% of changes are correct).

Default-off rationale and relation to the headline F1. The `--reflection` flag is *off* by default and was *not* used in the 500-sample four-way comparison reported in Section 7 (the F1 = 0.965 hybrid number is the single-call configuration). We treat reflection as an **opt-in additive enhancement**: it is enabled only when the operator wants to trade an estimated ∼7.6% extra LLM calls (only the IRRELEVANT verdicts among the ∼12.7% Stage B samples re-prompt) for the rescue/reversal behaviour quantified above. Keeping reflection off in the headline experiment ensures the F1 = 0.965 vs. F1 = 0.387 gap reflects *rules versus pure LLM*, not *rules+reflection versus pure LLM*; any improvement from reflection is therefore additive and deployment-time tunable rather than baked into the headline number.

5.7 Filter 3: Coverage Prediction

Filter 3 has two operational modes, selected automatically by inspecting the input CSV:

- **Label mode (default).** If the input already contains a `condition_cover_rate` column (e.g. the LessIsMore-FSE2025 `filter_train.csv`), `run_coverage_filter` performs an $O(N)$ row scan and removes rows whose coverage is below the threshold (default 0.01, matching the original paper). No model is loaded in this mode. All Filter 3 numbers in Section 7.7 come from this path.
- **Model mode (optional).** If the input does not carry a coverage label, the `cleantest-coverage-filter` skill exposes a `run_model_based_filter` entry point that loads a fine-tuned **Qwen2.5-Coder-0.5B** [17] regression checkpoint (default; any HuggingFace `*ForSequenceClassification` backbone with `num_labels=1` is supported via `--base_model`) and predicts coverage from the concatenated focal method + test case before applying the same threshold. Concretely:
  - Input: `f"{src_fm} [SEP] {target}"` truncated to 512 tokens.
  - Output: scalar predicted coverage $\in [0, 1]$.
  - Threshold: < 0.01 removed (following the original paper).
  - Fallback: filter is skipped gracefully if no model path or no GPU is provided.

  Held-out MAE / MSE / RMSE / Pearson r / Spearman ρ / threshold-sweep F1 for the trained Qwen2.5-Coder-0.5B checkpoint are reported in Section 7.7.3.

5.8 Data Flow Specification

Table 10 specifies the input/output contracts for each pipeline stage, enabling independent development and testing.

Each filter function follows a uniform contract: `run_X_filter(df, report, **config) -> List[int]`. This uniformity enables the Pipeline Pattern and makes it straightforward to insert new filter stages between existing ones.

5.9 Interface Design: LLM Interaction Protocol

The LLM interaction follows a strict request-response protocol with the following guarantees:

Table 10. Data Flow Specification: Per-Stage Input/Output Contracts

Stage	Input	Output	Side Effect
Load	CSV path (str)	DataFrame with `src_fm`, `target` columns	Validates schema
Deduplicate	DataFrame	DataFrame (subset)	Logs removed count
Filter 1	DataFrame + config	List[int] (indices to remove)	Updates NoiseReport
Filter 2	DataFrame + config	List[int] (indices to remove)	Updates NoiseReport; may call LLM API
Filter 3	DataFrame + config	List[int] (indices to remove)	Updates NoiseReport
Report	NoiseReport object	JSON file + MD file	Writes to disk

(1) **Input sanitization**: Code snippets are truncated to 1,500 characters maximum to stay within token limits. Non-UTF8 characters are stripped.
(2) **Output parsing**: The LLM response is parsed by checking the first word (case-insensitive). Any response not starting with the expected keyword defaults to the conservative choice (NOISE for syntax confirmation, IRRELEVANT for relevance judgment).
(3) **Timeout and retry**: Each API call has a 30-second timeout. On failure, the system retries up to 3 times with exponential backoff (2s, 4s, 8s). After 3 failures, the sample is classified using the rule-only verdict.
(4) **Determinism**: All LLM calls use `temperature=0.0` to ensure reproducible results across runs.
(5) **Cost control**: LLM calls are gated behind `--llm_enhance` flag and only triggered for borderline cases (12.7% of samples for relevance, ∼5% for syntax confirmation).

5.10 State Management

The pipeline maintains state through the `NoiseReport` dataclass, which accumulates statistics as samples flow through the pipeline:

Listing 5. NoiseReport state management

```
@dataclass
class NoiseReport:
    total_samples: int = 0
    removed_samples: int = 0
    kept_samples: int = 0
    breakdown: Dict[str, int] = field(default_factory=dict)
    llm_calls: int = 0
    llm_overrides: int = 0

    @property
    def removal_rate(self) -> float:
        if self.total_samples == 0:
```

```
            return 0.0
        return self.removed_samples / self.total_samples

    def add_noise(self, noise_type: str, count: int = 1):
        self.breakdown[noise_type] = (
            self.breakdown.get(noise_type, 0) + count
        )
        self.removed_samples += count

    def finalize(self):
        self.kept_samples = (
            self.total_samples - self.removed_samples
        )

    # Output adapters (defined in report_generator.py;
    # signatures shown for completeness):
    def to_json(self, path) -> Path: ...
    def to_markdown(self, path) -> Path: ...
```

This design ensures that reporting logic is decoupled from filtering logic — filters only call `report.add_noise()`, and the report handles all aggregation and output formatting internally.

5.11 Pipeline Orchestrator

The orchestrator skill coordinates the three filters:

6 Implementation

6.1 Technology Stack

Table 11. Technology Stack

Component	Technology
Language	Python 3.10+
AST Parsing	tree-sitter + tree-sitter-java
Pattern Matching	pyahocorasick (Aho-Corasick automaton)
Data Processing	pandas
LLM Client	OpenAI-compatible SDK (DeepSeek-V4-Flash)
Coverage Filter (model mode, optional)	PaddlePaddle 3.0 + PaddleNLP 3.0.0b4 (Python 3.10.10, CUDA 12.6) on a single NVIDIA A800-SXM4-80 GB hosted on Baidu PaddlePaddle AI Studio (the configuration used to produce the held-out numbers in Section 7.7.3); the same code path is also available as a PyTorch + HuggingFace Transformers variant for deployment on smaller GPUs. Default base model: Qwen2.5-Coder-0.5B.
Testing	pytest + pytest-cov
CI/CD	GitHub Actions (Python 3.10/3.11/3.12 matrix)
Linting	flake8 + mypy

Algorithm 1 CleanTest-Agent Pipeline

Require: Dataset D, config C (thresholds, LLM enable flag)
Ensure: Cleaned dataset D', noise report R

$D \leftarrow$ Deduplicate(D, key=`target`)
$R \leftarrow$ InitReport($|D|$)
▷ Stage 1: Syntax Noise Filter
for all $(f, t) \in D$ **do**
    $noise \leftarrow$ SyntaxFilter(f, t, C)
    if $noise \neq$ `null` **then**
        R.AddNoise($noise$)
        Remove (f, t) from D
    end if
end for
▷ Stage 2: Relevance Filter
for all $(f, t) \in D$ **do**
    $overlap \leftarrow$ ASTNameMatch(f, t)
    if $overlap = 0$ **then**
        if C.llm_enhance **then**
            $verdict \leftarrow$ LLMJudge(f, t)
            if $verdict =$ "RELEVANT" **then**
                continue
            end if
        end if
        R.AddNoise("no_relevance")
        Remove (f, t) from D
    end if
end for
▷ Stage 3: Coverage Filter (optional, two modes)
if not C.skip_coverage **then**
    if D has column `condition_cover_rate` **then**
▷ Label mode: read JaCoCo ground truth ($O(N)$)
        for all $(f, t) \in D$ **do**
            $cov \leftarrow D[(f, t)]$.`condition_cover_rate`
            if $cov < C$.threshold **then**
                R.AddNoise("low_coverage")
                Remove (f, t) from D
            end if
        end for
    else
▷ Model mode: predict coverage with the fine-tuned
▷ Qwen2.5-Coder-0.5B regression checkpoint, then apply
▷ the same threshold; skipped with a warning when
▷ neither labels nor a checkpoint are available.
    end if
end if
R.Finalize()
return D, R

6.2 Design Decisions

Several non-trivial design decisions were made during implementation:

Decision 1: Iterative vs. Recursive AST Traversal. The original CleanTest code uses recursive functions for AST traversal. During full-dataset testing, we encountered `RecursionError` on samples with deeply nested AST structures (some Java code generates ASTs with >1000 nesting depth). We converted all recursive traversals to iterative stack-based DFS, which is functionally equivalent but immune to stack overflow:

Listing 6. Iterative AST traversal for grammar error detection

```
def detect_grammar_errors(root_node) -> bool:
    stack = [root_node]
    while stack:
        node = stack.pop()
        # Skip leaves and string literals (avoid spurious matches)
        if len(node.children) == 0 or node.type == "string":
            continue
        if node.type == "ERROR":
            return True
        # A method declaration without a "block" child is incomplete
        if node.type == "method_declaration":
            child_types = [c.type for c in node.children]
            if "block" not in child_types:
                return True
        stack.extend(node.children)
    return False
```

Decision 2: Filter Priority Order. Our pipeline runs filters sequentially with Unnecessary Annotations first (highest volume, 43.68%), then AST-based checks, then relevance. A sample removed by Filter 1 is never processed by Filter 2, reducing total computation.

Decision 3: Lazy Aho-Corasick Construction. The automaton is expensive to build (∼0.1s) but fast to query. We construct it lazily on first use and cache it globally, avoiding repeated construction across multiple pipeline calls.

Decision 4: Graceful Degradation. Each sample is wrapped in `try/except`. LLM and coverage model are optional dependencies.

6.3 Core Algorithm Implementation

6.3.1 AST-Based Method Extraction. The relevance filter (Filter 2) requires extracting method declarations from focal methods and method invocations from test cases. We implement this using tree-sitter's concrete syntax tree traversal:

Listing 7. Extracting method declarations from focal method AST

```
def extract_src_methods(root_node) -> List[Tuple[str, int]]:
    """Extract␣(method_name,␣param_count)␣from
␣␣␣␣method␣declarations␣in␣the␣focal␣method."""
    methods = []
    stack = [root_node]
```

```
    while stack:
        node = stack.pop()
        if node.type == "method_declaration":
            name_node = node.child_by_field_name("name")
            params = node.child_by_field_name(
                "parameters"
            )
            if name_node:
                param_count = sum(
                    1 for c in (params.children if params
                    else [])
                    if c.type == "formal_parameter"
                )
                methods.append(
                    (name_node.text.decode(), param_count)
                )
        stack.extend(node.children)
    return methods
```

Listing 8. Extracting method invocations from test case AST

```
def extract_test_invocations(root_node)
    -> List[Tuple[str, int]]:
    """Extract (method_name, arg_count) from
    method invocations in the test case."""
    invocations = []
    stack = [root_node]
    while stack:
        node = stack.pop()
        if node.type == "method_invocation":
            name_node = node.child_by_field_name("name")
            args = node.child_by_field_name("arguments")
            if name_node:
                arg_count = sum(
                    1 for c in (args.children if args
                    else [])
                    if c.type not in ("(", ")", ",")
                )
                invocations.append(
                    (name_node.text.decode(), arg_count)
                )
        stack.extend(node.children)
    return invocations
```

6.3.2 Relevance Computation. The relevance score is computed as the intersection size between declared methods and invoked methods:

Listing 9. Relevance computation via (name, count) overlap

```
def compute_relevance(
    src_methods: List[Tuple[str, int]],
```

```
    test_invocations: List[Tuple[str, int]]
) -> int:
    """Compute overlap between focal-method
    declarations and test invocations.

    Returns the size of the intersection of
    (method_name, parameter_count) tuples.
    """
    return len(set(src_methods) & set(test_invocations))
```

When `compute_relevance() == 0`, the sample enters the LLM judgment stage (Stage B). This two-stage design ensures that the expensive LLM call is only made for the ∼12.7% of samples where the fast rule-based check is inconclusive.

6.3.3 Aho-Corasick Automaton Construction. The annotation pattern dictionary is loaded once and cached globally:

Listing 10. Lazy Aho-Corasick automaton construction

```
_AC_AUTOMATON = None

def _open_noise_modifier_file():
    """Locate the dictionary as package data first, then fall back
    to the repository layout for development checkouts."""
    try:
        from importlib.resources import files
        return (files("cleantest_agent.data")
                / "noise_modifier_fm.txt").open("r")
    except (ModuleNotFoundError, FileNotFoundError):
        repo = Path(__file__).resolve().parent.parent
        fallback = (repo / "skills" / "cleantest-syntax-filter"
                    / "references" / "noise_modifier_fm.txt")
        return fallback.open("r") if fallback.exists() else None

def _load_noise_modifiers():
    """Build Aho-Corasick automaton (lazy, cached)."""
    global _AC_AUTOMATON
    if _AC_AUTOMATON is not None:
        return _AC_AUTOMATON

    fh = _open_noise_modifier_file()
    if fh is None:
        logger.warning("Noise dictionary not found; "
                       "Unnecessary Annotations filter disabled.")
        return None

    import ahocorasick
    automaton = ahocorasick.Automaton()
    with fh as f:
        for idx, line in enumerate(f):
            pattern = line.strip("\n")
```

```
            if pattern:
                automaton.add_word(pattern, (idx, pattern))
    automaton.make_automaton()
    _AC_AUTOMATON = automaton
    return _AC_AUTOMATON
```

After construction, each sample is checked in $O(L)$ time (where L is the source code length) by iterating over the automaton:

Listing 11. O(L) annotation noise detection

```
def _has_unnecessary_annotations(source_code: str)
    -> bool:
    automaton = _load_noise_modifiers()
    if automaton is None:           # graceful degradation
        return False
    for _ in automaton.iter(source_code):
        return True  # First match is sufficient
    return False
```

6.3.4 Input Data Validation. The data loader performs strict validation before pipeline execution:

Listing 12. Input CSV validation

```
REQUIRED_COLUMNS = {"src_fm", "target"}

def load_csv(path: Union[str, Path]) -> pd.DataFrame:
    """Load␣and␣validate␣input␣CSV.

␣␣␣␣Raises␣FileNotFoundError␣if␣the␣file␣is␣missing,
␣␣␣␣ValueError␣if␣the␣suffix␣is␣wrong␣or␣required
␣␣␣␣columns␣are␣absent.
␣␣␣␣"""
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(
            f"Dataset␣not␣found:␣{path}"
        )
    if path.suffix != ".csv":
        raise ValueError(
            f"Expected␣CSV␣file,␣got:␣{path.suffix}"
        )

    df = pd.read_csv(path, low_memory=False)
    missing = REQUIRED_COLUMNS - set(df.columns)
    if missing:
        raise ValueError(
            f"Missing␣required␣columns:␣{missing}.␣"
            f"Available:␣{list(df.columns)}"
        )
    return df
```

This fail-fast approach ensures that malformed input is caught immediately with a descriptive error message, rather than causing cryptic failures deep in the pipeline.

6.4 Design Patterns Applied

We apply four classical (Gang-of-Four–style) design patterns from the software engineering literature that directly address our system's architectural requirements. A fifth, agentic-style pattern (Reflection, Lab 1) is documented separately in Section 8 as a method improvement that ships inside the relevance filter's LLM stage.

6.4.1 Strategy Pattern (Filter Selection). Each filter implements a common interface: given a sample (f, t), return either a noise classification or `null` (clean). The pipeline orchestrator delegates to each filter strategy without knowing implementation details:

Listing 13. Strategy Pattern: Filter interface

```
class FilterStrategy(Protocol):
    def detect(self, src_fm: str, target: str,
               config: dict) -> Optional[str]:
        """Returns noise type or None if clean."""
        ...
```

This allows new filter strategies to be added (e.g., a future "flaky test" detector) without modifying the orchestrator — satisfying the Open-Closed Principle (OCP).

6.4.2 Pipeline Pattern (Sequential Processing). The orchestrator implements the Pipeline Pattern: each filter stage processes the remaining samples sequentially, with early termination when a sample is flagged as noise. This order-dependent design is critical because:

- **Annotation noise is checked first** (43.68% of all noise): removing these samples early reduces the input to subsequent, more expensive filters.
- **LLM-based relevance judgment is last** among rule stages: only samples that pass all rule checks may trigger an LLM call, minimizing API cost.

6.4.3 Facade Pattern (LLM Client). The `llm_client.py` module provides a simplified facade over the OpenAI-compatible API, hiding:

- Client initialization and lazy caching (singleton pattern internally)
- Prompt template formatting
- Response parsing (extracting "NOISE"/"KEEP" from free-text responses)
- Retry logic and error handling

Callers simply invoke `llm_confirm_syntax_noise(code, noise_type, rule_result)` without concern for API mechanics. This decouples filter logic from LLM provider details, allowing straightforward switching between providers (GPT-4o-mini, DeepSeek, Claude).

6.4.4 Observer Pattern (Report Generation). The `NoiseReport` dataclass acts as an accumulator that observes the pipeline's progress. Each time a filter removes a sample, it notifies the report via `report.add_noise(noise_type)`. After pipeline completion, `report.finalize()` computes summary statistics and generates both JSON and Markdown outputs. This separation ensures pipeline logic is not polluted with reporting concerns.

6.5 Project Structure

The codebase is organized as follows:

Listing 14. Repository structure

```
cleantest-agent/
  skills/
    cleantest-pipeline/SKILL.md
    cleantest-syntax-filter/SKILL.md
    cleantest-relevance-filter/SKILL.md
    cleantest-coverage-filter/SKILL.md
  cleantest_agent/              # Installable Python package
    __init__.py         # Public API
    pipeline.py         # Main orchestrator
    parser_utils.py     # AST detection functions
    llm_client.py       # LLM API wrapper
    data_loader.py      # CSV I/O + validation
    report_generator.py# Report output
    data/
      noise_modifier_fm.txt  # 21,954-pattern dict (package data)
  tests/                # 36 test cases
  experiments/           # Baseline comparison
  pyproject.toml        # Package metadata
  .github/workflows/ci.yml
```

6.6 Skill Installation

Skills can be installed globally or per-project:

Listing 15. Installing skills to CodeBuddy

```
# Global install
make install
# Copies skills/* to ~/.codebuddy/skills/

# Project-level install
cp -R skills/* .codebuddy/skills/
```

After installation, the coding assistant automatically discovers the skills and invokes them when triggered by natural language queries such as “clean test data” or “filter noisy tests.”

6.7 Error Handling

The pipeline includes defensive mechanisms:

- **try/except per sample**: Malformed code that causes parser errors is skipped rather than crashing the pipeline.
- **Iterative AST traversal**: All tree-sitter traversals use stack-based DFS instead of recursion to avoid `RecursionError` on deeply nested code.
- **Graceful degradation**: If the coverage model or LLM API is unavailable, those stages are skipped with a warning.

6.8 Testing Strategy

We implement 36 test cases across 5 test files:

Table 12. Test Coverage

Test File	Cases	Focus
`test_syntax_filter.py`	16	All syntax noise types + annotations
`test_relevance_filter.py`	7	AST extraction + overlap
`test_coverage_filter.py`	4	Threshold + graceful skip
`test_pipeline.py`	4	End-to-end integration
`test_report.py`	5	Report generation
Total	**36**	All passing

7 Experimental Evaluation

7.1 Research Questions

- **RQ1**: How does CleanTest-Agent's noise detection accuracy compare to the original CleanTest?
- **RQ2**: How does the model-driven approach (rules + LLM) compare to pure LLM approaches?
- **RQ3**: What is the performance impact of the Aho-Corasick optimization?
- **RQ4**: What is the cost-effectiveness of each approach?

7.2 Dataset

We use the Methods2Test training dataset:

- **Original size**: 624,022 rows
- **After deduplication**: 593,953 unique samples
- **Columns**: `src_fm` (focal method), `target` (test case)
- **Source**: Zenodo DOI 10.5281/zenodo.15347368
- **Input file SHA-256**: `2a1af335...0c18836`
- **File**: `dataset/training_dataset/all_dataset/all_train.csv` from the LessIsMore-FSE2025 replication package.

7.2.1 Reproduction Setup. All full-pipeline numbers reported in this section come from a single reproducible run, archived under `experiments/results/full_run/` in the project repository. Key metadata:

- **Hardware**: Apple M3, 16 GB unified memory, macOS 24.6 (arm64).
- **Software**: Python 3.9.6, pandas 2.3.3, tree-sitter, tree-sitter-java, pyahocorasick, tqdm 4.67.1.
- **Reproduction command**:

```
python3 -m cleantest_agent.pipeline \
    --input_csv path/to/all_train.csv \
    --output_dir experiments/results/full_run \
    --skip_coverage
```

- **Wall-clock time**: 158.04 s (2 min 38 sec) for the full Filter 1 + Filter 2 pipeline on 593,953 deduplicated samples.
- **Filter 3** is invoked separately on the labelled `filter_train.csv` subset (469,174 samples with `condition_cover_rate`); see Section 7.7.

The archived run directory contains: `run_log.txt` (full execution log), `noise_report.json` (per-noise-type counts), `summary.md` (human-readable report), `env_info.md` (environment metadata), and `filtered_data.sample.csv` (the first 1,000 retained samples; the full 273,383-row `filtered_data.csv` is regenerated by re-running the command above).

7.3 RQ1: Noise Detection Accuracy

Table 13 compares our system's noise detection results with the original CleanTest paper.

Table 13. Noise Detection: CleanTest-Agent vs. Original Paper

Noise Type	**Ours**	**Paper**	**Match**
Unnecessary Annotations	259,427 (43.68%)	— (41.64%)	≈
No Relevance	29,885 (5.03%)	— (12.70%)	*
Syntax Error	16,372 (2.76%)	— (1.11%)	**
Ambiguous Data Type	10,335 (1.74%)	— (3.08%)	≈
Empty Exception	2,259 (0.38%)	— (0.68%)	≈
Non-English Literal	1,479 (0.25%)	— (0.16%)	≈
Missing Implementation	813 (0.14%)	— (0.34%)	≈
Total Removed	**320,570 (53.97%)**	— (43.52%)	

* Our pipeline checks annotation noise first (highest priority), removing many samples before the relevance check.
** We check both `src_fm` and `target` for syntax errors; the original only checks `src_fm`.
Note on counting semantics: The two columns of percentages are not strictly comparable because they use different denominators *and* different counting rules. The original paper's per-type percentages (Table 5 of Zhang et al. [39]) are computed against the full 780,944-sample Methods2Test dataset and a sample contributing to two noise types is counted once per type, so the eight types sum to 63.66 % — larger than the 43.52 % of unique samples that contain at least one noise type. Our per-type counts use the deduplicated 593,953-sample subset as the denominator and are *disjoint* by construction (a sample removed by Filter 1 never reaches Filter 2 or 3 due to priority ordering), so our seven types sum exactly to the 53.97 % of removed samples. Direct column-vs-column comparison is therefore meaningful only for the single largest type (Unnecessary Annotations, where the priority choice does not interfere) and for the order-of-magnitude check on total noise prevalence.

The Unnecessary Annotations ratio (43.68 % vs. 41.64 %) is close, confirming that our Aho-Corasick implementation correctly reproduces the original dictionary matching against a deduplicated denominator (593,953 vs. 780,944 samples). Differences in other categories are mostly explained by the priority-ordering effect documented above (the original paper allows the same sample to be flagged for multiple types; ours assigns each removed sample to its first-matching type), and by the input-side difference for syntax errors (we scan both `src_fm` and `target`; the original only scans `src_fm`).

7.4 RQ2: Model-Driven vs. Pure LLM

We compare four approaches on a stratified sample of 500 rows.

Note on methodology: The rule-based system labels are computed on actual data and serve as ground truth. The LLM baseline results are obtained by running DeepSeek-V4-Flash (via Tencent Cloud TokenHub) on a stratified sample of 500 rows. The experiment script (`experiments/run_baselines.py`) is included in our repository for reproducibility.

Key findings:

Table 14. Comparison: Model-Driven vs. Pure LLM Approaches

Method	Prec.	Recall	F1	Time (s)
Rule-based only	1.000	1.000	1.000	0.11
LLM zero-shot (DeepSeek-V4-Flash)	0.505	0.221	0.307	1,487
LLM few-shot (DeepSeek-V4-Flash)	0.534	0.303	0.387	1,642
Hybrid (ours)†	**0.974**	**0.956**	**0.965**	**<60**

† Hybrid F1 is computed from the ablation study (Table 27): rules provide the base detection, and LLM enhances borderline cases. The 0.965 reflects rule–LLM disagreements where LLM overrides the rule verdict on ∼5% of samples.

(1) Pure LLM zero-shot achieves only F1=0.307, primarily due to extremely low recall (0.221). The LLM struggles to detect annotation noise because it lacks the 21,954-pattern dictionary and tends to classify annotated code as "normal Java."
(2) Few-shot improves recall to 0.303 and F1 to 0.387, but still falls far short of the rule-based approach. Providing examples helps slightly but cannot compensate for the fundamental dictionary knowledge gap.
(3) Our hybrid approach achieves F1=0.965 by using rules for high-confidence detection and LLM only for the ∼12.7% of borderline cases.
(4) The rule-based pipeline processes 500 samples in **0.11 seconds** compared to 1,487 seconds for zero-shot and 1,642 seconds for few-shot — a **13,000–15,000× speedup**.

7.4.1 Confusion Matrix Analysis. Table 15 presents the detailed confusion matrices for both LLM baselines.

Table 15. Confusion Matrices for LLM Baselines (500 samples; total noise=231, clean=269)

	TP	FP	FN	TN
Zero-shot	51	50	180	219
Few-shot	70	61	161	208

The most striking finding is the **false negative rate**: 77.9% for zero-shot (180/231) and 69.7% for few-shot (161/231). This means the LLM *misses* the vast majority of noisy samples, classifying them as clean. The primary failure mode is not false alarms (FP is moderate) but catastrophic under-detection.

7.4.2 Error Analysis by Noise Type. To understand *which* noise types the LLM fails on, we analyze the 180 false negatives from the zero-shot experiment:

The data confirms that **annotation noise accounts for 87.8% of all false negatives** (158/180). This single noise type — which requires matching against a domain-specific dictionary — is the dominant reason pure LLM approaches fail. The LLM performs relatively better on relevance noise where semantic reasoning is applicable, but these account for only a small fraction of total noise.

This finding directly validates our architectural decision: use Aho-Corasick (deterministic, 100 % recall *against the encoded dictionary patterns*) for annotation detection, and reserve LLM for the semantic tasks (relevance judgment) where it adds genuine value.

7.4.3 Few-Shot vs. Zero-Shot: Marginal Improvement. Few-shot provides only marginal improvement over zero-shot:

Table 16. LLM Zero-Shot False Negative Distribution by Noise Type

Noise Type	Total	FN	Miss Rate	Root Cause
Unnecessary Annotations	191	158	82.7%	LLM lacks 21,954-pattern dictionary
Syntax Error	8	4	50.0%	LLM partially recognizes broken code
Non-English	7	4	57.1%	LLM misses CJK characters
Ambiguous Type	7	4	57.1%	Subtle pattern, LLM misses most
No Relevance	15	7	46.7%	LLM sometimes catches irrelevance
Missing Implementation	2	2	100%	LLM sees minimal code as valid
Empty Exception	1	1	100%	Single sample, too few to generalize
Total	**231**	**180**	**77.9%**	

Sub-totals are illustrative: the rule-based labeller assigns each sample a single dominant noise type; the per-type breakdown above sums to 231 by construction. Miss rates are computed within each row, against that row's total.

- Recall increases from 22.1% to 30.3% (+8.2 percentage points)
- Precision increases slightly from 50.5% to 53.4%
- F1 improves from 0.307 to 0.387 (+26% relative, but still far below acceptable levels)

The few-shot examples help the LLM recognize *common* annotation patterns (`@GetMapping`, `@ApiOperation`) that appear in the examples, but cannot generalize to the long tail of 21,954 patterns. This demonstrates a fundamental limitation: few-shot prompting cannot substitute for exhaustive pattern knowledge.

7.5 RQ3: Aho-Corasick Performance

Table 17. Performance: Naïve vs. Aho-Corasick Matching

Metric	Naïve	Aho-Corasick	Speedup
Filter 1 time	∼30 min	∼1.6 min	18.8×
Full pipeline	∼31 min	∼2.6 min	11.5×
Throughput	∼330 samples/s	∼3,800 samples/s	11.5×
Results	53.97% removed	53.97% removed	Identical

The Aho-Corasick automaton construction takes <0.1 seconds (one-time cost) and reduces per-sample annotation matching from $O(K \times L)$ to $O(L + Z)$, where $K = 21{,}954$, L is text length, and Z is the number of matches (typically ≤ 1).

7.6 RQ4: Cost-Effectiveness Analysis

The rule-based pipeline is **13,000–15,000× faster** than pure LLM approaches. Even the hybrid approach (which uses LLM for only ∼12.7% of borderline cases) completes in under 60 seconds — 25× faster than the pure LLM alternatives.

7.6.1 Token Consumption Analysis. Based on our 500-sample experiment, we estimate the per-sample token consumption:

Table 18. Cost-Effectiveness per 500 Samples (Real Experiment)

Method	F1	Time (s)	ms/sample
Rule-based	1.000	0.11	0.2
LLM zero-shot	0.307	1,487	2,975
LLM few-shot	0.387	1,642	3,283
Hybrid	**0.965**	**<60**	**<120**

Table 19. Estimated Token Consumption per Sample

Method	Input Tokens	Output Tokens	Total/Sample
Zero-shot	∼400	∼50	∼450
Few-shot	∼700	∼50	∼750
Hybrid (12.7% LLM)	∼50 (amortized)	∼6	∼56

7.6.2 Full-Scale Cost Projection. Extrapolating from our 500-sample experiment to the full Methods2Test dataset (593,953 samples):

Table 20. Projected Cost and Time for Full Dataset (593,953 Samples)

Method	API Calls	Time	API Cost[1]	F1
Rule-based	0	2.6 min	$0	1.000
Zero-shot	593,953	∼20 days	∼$35	0.307
Few-shot	593,953	∼25 days	∼$58	0.387
Hybrid	∼75,432 (12.7%)	∼3.5 hours	∼$4.5	**0.965**

At full scale, the pure LLM approach would require **20–25 days of continuous API calls** to process the entire dataset (at the observed rate of ∼2.9 seconds per call). This makes pure-LLM approaches not only expensive but operationally infeasible for large-scale data cleaning tasks. Our hybrid approach reduces this to 3.5 hours by limiting LLM calls to only borderline cases, while the rule-based core completes in just 2.6 minutes.

7.7 Filter 3 Validation on Real Coverage Labels

To validate Filter 3 on real data, we run `run_coverage_filter` on `filter_train.csv` from the LessIsMore-FSE2025 replication package (469,174 samples, each carrying a `condition_cover_rate` label produced by JaCoCo on the original CleanTest pipeline).

Scope of this validation. The label-mode results in this section come from a deterministic row-scan over the ground-truth `condition_cover_rate` column (Section 4.5); no model is invoked there. The model-mode results in Section 7.7.3 below come from a *trained* Qwen2.5-Coder-0.5B checkpoint produced on the same data on a single NVIDIA A800 80 GB; held-out MAE / MSE / Pearson r / Spearman ρ / threshold-aware F1 numbers are reported there.

[1]Based on DeepSeek-V4-Flash pricing: ¥0.8/1M input tokens, ¥2/1M output tokens.

7.7.1 Coverage Label Distribution. Table 21 summarises the empirical distribution of the ground-truth coverage labels.

Table 21. Coverage Label Statistics on `filter_train.csv` (469,174 samples)

Statistic	Value
min	0.011
25%	0.063
median	0.108
mean	0.127
75%	0.163
max	1.910
std	0.101

A first observation is that the empirical minimum is exactly $0.011 > 0.01$: no sample in this file has `condition_cover_rate` below the default threshold. This independently confirms the hypothesis that `filter_train.csv` is the *post-CleanTest* output produced with the paper's threshold of 0.01 — i.e. samples below 0.01 had already been removed during dataset construction. We therefore treat this file as the "Filter 3 baseline output" rather than as raw input.

7.7.2 Threshold Sensitivity Sweep. Even though the paper's default threshold of 0.01 cannot remove any sample from this already-filtered file, the threshold-sensitivity sweep in Table 22 demonstrates that `run_coverage_filter` correctly applies any user-supplied threshold on real data, and quantifies the trade-off between data purity and data retention.

Table 22. Filter 3 Threshold Sensitivity (469,174 samples; Apple M3 / 16 GB)

Threshold	Removed	Removed %	Kept	Time (s)	Throughput
0.010	0	0.00%	469,174	6.55	71.7K samples/s
0.050	84,021	17.91%	385,153	6.43	72.9K samples/s
0.100	212,059	45.20%	257,115	6.42	73.0K samples/s
0.150	327,016	69.70%	142,158	6.41	73.2K samples/s
0.200	400,999	85.47%	68,175	6.43	72.9K samples/s
0.300	448,842	95.67%	20,332	6.49	72.3K samples/s

Key observations:

(1) **Correctness on real data**: `run_coverage_filter` processes 469,174 labelled samples in ∼6.4 seconds at ∼73K samples per second, with no errors and no skipped rows. This closes the loop on Filter 3 — it now has end-to-end execution evidence on real data, complementing the unit tests in `tests/test_coverage_filter.py`.

(2) **Threshold = 0.01 reproduces the paper**: removing 0/469,174 samples means our implementation matches the original CleanTest's threshold semantics exactly (strict inequality `coverage` $< t$).

(3) **Threshold guidance for downstream users**: a researcher looking for an aggressive cleaning regime can set $t = 0.10$ and retain only the 257K samples with non-trivial coverage; a researcher

prioritising data volume can keep all 469K. This is exactly the "configurable aggressiveness knob" discussed in Section 8 as a proposed three-tier classification.

(4) **Linear scaling**: throughput is essentially constant (∼73K samples/s) across all thresholds, confirming that the filter's cost is dominated by row iteration and label comparison, not by the threshold value itself. This is the expected $O(N)$ behaviour.

The full run log, threshold-sweep JSON, and reproduction script are archived under `experiments/results/coverage_run/`.

7.7.3 Model-Mode Validation: Fine-Tuned Qwen2.5-Coder-0.5B. We additionally validate Filter 3's *model-mode* code path by fine-tuning Qwen2.5-Coder-0.5B (500 M parameters, Alibaba Tongyi Lab, [17]) on the same `filter_train.csv` (469,174 rows) and reporting held-out regression metrics.

Why we need a model-mode at all. The label-mode path validated above (Section 7.7) is sufficient *only* when the input CSV carries a ground-truth `condition_cover_rate` column produced by an expensive offline measurement step — in the LessIsMore-FSE2025 pipeline this requires (i) compiling each test case against its focal-method module, (ii) executing the compiled tests through JaCoCo [12], and (iii) extracting per-test branch coverage counts. This entire chain takes hours per project and is the main reason `filter_train.csv` is delivered as a one-shot artefact rather than computed on demand. In the much more common *label-free* scenario — a researcher with an unlabelled CSV of `(src_fm, target)` pairs scraped from a new project — the Filter 3 pipeline must produce a usable coverage proxy *without* running JaCoCo. This is the operational niche that model mode fills, and it is the reason the original CleanTest paper [39] also trained a CodeGPT regression head as part of its release.

Our experiment answers two concrete questions: (Q1) does a *small*, modern code-specific backbone (0.5 B parameters, code pre-training) achieve usable regression quality on Methods2Test, given a fixed single-GPU compute budget? (Q2) how does it compare to the published CodeGPT baseline that ships with the original CleanTest paper?

Target variable. The regression target `condition_cover_rate` is the JaCoCo *branch coverage ratio* for the focal method when its paired test case is executed: the number of branch outcomes (*true*/*false* on each conditional) reached by the test, divided by the total number of branch outcomes in the focal method. It is bounded in $[0, 1]$ in principle; we observe `min` $= 0.011$, `max` $= 1.91$ in the raw labels (305 rows above 1.0, attributable to JaCoCo's handling of nested short-circuit operators) and clip during training. The empirical distribution on the full 469,174 rows has `mean` $= 0.127$, `median` $= 0.108$, `std` $= 0.101$ (Table 21), so the task is a *low-variance, right-skewed regression* concentrated below 0.30.

Setup. The data is split 80/10/10 (train / valid / test) stratified by quintile of `condition_cover_rate` so that all three splits preserve the empirical coverage distribution (`prepare_data.py` performs this split with seed 42 to make the result fully reproducible). The base model is loaded via `Qwen2ForSequenceClassification` with `num_labels=1`, the input is `f"{src_fm} [SEP] {target}"` truncated to 512 tokens, and the regression head is wrapped with a sigmoid before the MSE loss so that predictions remain inside the same $[0, 1]$ interval as the (clipped) training labels. Training uses AdamW with cosine warmup, learning rate 3×10^{-5}, weight decay 0.01, bf16 mixed precision, an effective train batch size of 64 (per-device 64 without gradient accumulation), 2 epochs, and validation-MSE early stopping (patience 3). Training runs on a single NVIDIA A800-SXM4-80 GB (Compute Capability 8.0, 79.3 GB usable HBM) hosted on Baidu PaddlePaddle AI Studio, with the software stack PaddlePaddle 3.0 + PaddleNLP 3.0.0b4 (`aistudio_sdk` 0.2.5, Python 3.10.10, CUDA 12.6 runtime under driver 12.8); the launch entry point is `skills/cleantest-coverage-filter/scripts_paddle/train_model.py`, invoked

from `experiments/main-final.ipynb`. The full split sizes are 375,338 train / 46,915 valid / 46,921 test (each $\sim 80\,/\,10\,/\,10\,\%$ of the 469,174 deduplicated rows in `filter_train.csv`).

Evaluation metrics. We report six standard regression metrics, chosen to expose complementary failure modes: **MAE** (the average absolute prediction error, in the same unit as the label), **MSE** (the loss the optimiser actually minimises, included so the training-versus-test comparison is on the optimiser's own yardstick), **RMSE** (interpretable in label units like MAE but penalises large errors more than small ones), R^2 (the fraction of label variance explained by the model; $R^2 = 0$ means "no better than always predicting the mean", $R^2 = 1$ means perfect predictions), **Pearson** r (linear correlation), and **Spearman** ρ (rank correlation, robust to monotonic nonlinearities). Spearman ρ matters specifically for Filter 3 because the downstream removal decision is a thresholded comparison — the model only needs to rank samples correctly relative to τ, not predict the exact coverage. The threshold-aware classification metrics in Table 25 operationalise this rank-only requirement.

Training dynamics. Table 23 shows the validation metrics at every eval checkpoint logged by the in-script `JsonlLogCallback`. Validation MSE decreases monotonically across all six checkpoints with no late-epoch divergence, and the train-to-test MAE gap is $|0.0304 - 0.0309| = 0.0005$ — essentially zero — so we do not see meaningful overfitting under this recipe.

Table 23. Filter 3 Model-Mode Validation Trajectory (per eval checkpoint, 46,915 validation samples)

Step	Epoch	val MSE	val MAE	val RMSE	Pearson	Spearman
2,000	0.34	0.00509	0.0403	0.0714	0.6935	0.7685
4,000	0.68	0.00435	0.0349	0.0659	0.7473	0.8209
6,000	1.02	0.00395	0.0324	0.0628	0.7767	0.8386
8,000	1.36	0.00372	0.0314	0.0610	0.7902	0.8457
10,000	1.71	0.00365	0.0304	0.0604	0.7927	0.8477
11,730	**2.00**	**0.00365**	**0.0304**	**0.0604**	**0.7927**	**0.8477**

Source: `experiments/results/coverage_run/metrics.jsonl` (filtered to `kind = "eval"` entries). The final row is the checkpoint selected for held-out evaluation in Table 24.

Held-out test metrics. Table 24 reports the regression metrics on the 10 % held-out test split (46,921 samples). All numbers come from a single training run archived under `experiments/results/coverage_run/` and can be regenerated end-to-end from `experiments/main-final.ipynb`.

Interpretation: is MAE 0.031 "good"? On a target with mean 0.127 and standard deviation 0.101, a held-out MAE of 0.031 corresponds to a relative error of $\approx 24.4\,\%$ of the label mean and only ≈ 0.31 standard deviations. Spearman $\rho = 0.848$ means that pairwise ranking of two random test cases by predicted coverage agrees with the JaCoCo-measured ranking on roughly 92 % of pairs (by the standard $\rho \to$ Kendall-τ conversion). For Filter 3's operational role — thresholded removal of low-coverage tests — this is the relevant quality criterion, and the threshold sweep below makes the trade-off explicit.

Threshold-aware low-coverage detection. The original CleanTest paper applies a fixed threshold $\tau = 0.01$ on the `condition_cover_rate` label to decide whether a sample should be removed. Because `filter_train.csv` is the *post-CleanTest* output (Section 7.7), no test sample has $y_{\text{true}} < 0.01$, which makes $\tau = 0.01$ degenerate (no positives to recover). We therefore also report a sweep over operationally

Table 24. Filter 3 Model-Mode Held-Out Test Metrics (Qwen2.5-Coder-0.5B; 46,921 samples)

Metric	Value
Regression on continuous coverage	
MAE	0.0309
MSE	0.0039
RMSE	0.0628
R^2	0.6041
Pearson r	0.7780
Spearman ρ	0.8481

meaningful thresholds in Table 25: at $\tau = 0.10$ (45.4 % of test rows are positives) the model achieves an F1 of 0.857, and at $\tau = 0.15$ it reaches 0.912. This demonstrates that the regressor's predictions are well-calibrated for the threshold range that practitioners actually choose.

Table 25. Filter 3 Model-Mode Threshold Sweep on the Held-Out Test Split (46,921 samples)

τ	Pos. %	Precision	Recall	F1
0.010	0.0 %	—	—	—
0.030	8.2 %	0.578	0.244	0.343
0.050	17.9 %	0.776	0.607	0.681
0.075	31.2 %	0.838	0.769	0.802
0.100	**45.4 %**	**0.880**	**0.835**	**0.857**
0.125	58.3 %	0.905	0.869	0.887
0.150	69.6 %	0.924	0.900	0.912
0.200	85.5 %	0.950	0.948	0.949

"Pos. %" is the fraction of held-out rows whose ground-truth coverage falls below τ. The $\tau = 0.01$ row is degenerate (no positives in the post-CleanTest test split, so precision/recall/F1 are undefined). Numbers are computed from `experiments/results/coverage_run/test_pred_a800.csv` and archived under `experiments/results/coverage_run/test_threshold_sweep.json`.

Failure analysis: where does the model under-perform? A single scalar MAE hides systematic biases. We bin the 46,921 held-out predictions by ground-truth coverage and report bin-wise MAE and signed bias (mean residual, $\bar{p} - \bar{y}$) in Table 26. Three observations follow:

(1) **The middle of the distribution is the most accurate**: bins $[0.05, 0.20)$ contain 67.6 % of the test rows and all have MAE ≤ 0.023.

(2) **The model exhibits regression-to-the-mean**: it over-predicts low-coverage rows ($+0.027$ bias on $[0.011, 0.05)$) and under-predicts the long tail of high-coverage rows (-0.153 bias on $[0.30, 1.91]$, only 4.3 % of the test set). This is the canonical behaviour of an MSE-trained regressor on a right-skewed target with few high-value training examples, and is not specific to our backbone choice.

(3) **Operational implication**: because Filter 3 removes *low*-coverage samples, the only bias direction that matters is the $+0.027$ over-prediction on $[0.011, 0.05)$ — and at the practitioner-preferred

threshold $\tau = 0.10$ the F1 is already 0.857 (Table 25), so this bias does not materially hurt the deployment metric.

Table 26. Filter 3 Model-Mode Bin-Wise Residuals on the Held-Out Test Split (46,921 samples)

Bin (by y_{true})	n	%	$\bar{y}$	$\bar{p}$	**MAE**	**bias**
$[0.011, 0.05)$	8,390	17.9	0.031	0.057	0.029	+0.027
$[0.05, 0.10)$	12,901	27.5	0.075	0.085	0.023	+0.010
$[0.10, 0.15)$	11,384	24.3	0.123	0.127	0.023	+0.004
$[0.15, 0.20)$	7,422	15.8	0.172	0.170	0.023	−0.002
$[0.20, 0.30)$	4,807	10.2	0.235	0.220	0.032	−0.015
$[0.30, 1.91]$	2,017	4.3	0.460	0.307	0.166	−0.153

Computed from `experiments/results/coverage_run/test_pred_a800.csv`; exact numbers archived in `experiments/results/coverage_run/eval_trajectory.json` (`residual_by_coverage_bin`).

Comparison with the original CleanTest CodeGPT.. The original CleanTest paper [39] reports MAE 7.98 % (i.e. ≈ 0.080 on the $[0, 1]$ scale) and MSE 1.05 % (≈ 0.0105 on the $[0, 1]$ scale) for its CodeGPT-based coverage regression. Our fine-tuned Qwen2.5-Coder-0.5B reduces MAE to **0.0309** (a $\approx 2.6\times$ reduction) and MSE to **0.0039** (a $\approx 2.7\times$ reduction) on the same data and the same regression target. We attribute the gain to (i) a stronger code-specific backbone (Qwen2.5-Coder pre-training vs. the older GPT-2–based CodeGPT) and (ii) a sigmoid-bounded regression head, which prevents the optimiser from chasing extrapolations outside the empirical $[0, 1.91]$ label range during training. We acknowledge one methodological caveat: the original paper's CodeGPT MAE was measured on its own training-test split, not on our 80/10/10 stratified split. The magnitude of our improvement ($\approx 2.6\times$) substantially exceeds plausible split variance, but a strict apples-to-apples comparison would require re-training CodeGPT on our exact split — this is the second follow-up listed in Section 9, Future Work.

Conclusion. The model-mode validation answers Q1 affirmatively (a 0.5 B code-specific backbone delivers MAE 0.031 / Spearman 0.85 with about 3.3 h of single-A800 training, which is well inside practical compute budgets) and Q2 quantitatively (a $\approx 2.6\times$ MAE improvement over the published CodeGPT baseline). Combined with the label-mode results above (Section 7.7), this validates Filter 3 in both of its two operational regimes — whenever JaCoCo labels are available, label mode is exact and free; when they are not, model mode produces a coverage proxy that is good enough to support the same threshold-based removal decision.

Reproducibility. Training takes **11,951 s** ($\approx$ 3.32 h) of wall-clock time on a single A800 80 GB. The full training log (`train_a800.log`), per-step / per-eval JSONL stream (`metrics.jsonl`), training summary (`training_metrics.json`), held-out predictions (`test_pred_a800.csv`), held-out scalar metrics (`test_metrics.json`), threshold sweep (`test_threshold_sweep.json`) and the extracted trajectory & bin-wise residual table (`eval_trajectory.json`) are archived under `experiments/results/coverage_run/`. The end-to-end notebook that produced all of these artefacts is `experiments/main-final.ipynb`.

7.8 Case Study: Walking Through Real Samples

To provide concrete intuition for why different methods succeed or fail, we present three representative samples from the Methods2Test dataset.

Listing 16. Case 1: Focal method with @ApiOperation annotation

7.8.1 Case 1: Unnecessary Annotation (Detected by Rules, Missed by LLM Zero-Shot).

```
@ApiOperation(value = "Get all users",
    notes = "Returns a list of all registered users")
@GetMapping("/api/users")
public List<User> getAllUsers() {
    return userRepository.findAll();
}
```

Rule-based: Immediately flagged as "unnecessary_annotations" by Aho-Corasick matching `@ApiOperation(value = "Get all users"...` against the noise dictionary. Time: <0.01 ms.

LLM zero-shot: Classified as "CLEAN". The LLM sees `@ApiOperation` and `@GetMapping` as standard Spring annotations and does not recognize them as noise for the test generation training task. It would need specific instructions about what constitutes "unnecessary" in this domain.

LLM few-shot: Correctly classified as "NOISE" after seeing a similar example in the few-shot prompt. This demonstrates that few-shot can improve LLM performance on domain-specific patterns, but at the cost of longer prompts and higher token consumption.

Listing 17. Case 2: Focal method

7.8.2 Case 2: Irrelevant Test (Detected by AST Matching, Confirmed by LLM).

```
public void saveUser(User user) {
    userRepository.save(user);
}
```

Listing 18. Case 2: Paired test (irrelevant)

```
@Test
public void testStringLength() {
    String s = "hello";
    assertEquals(5, s.length());
}
```

Rule-based (AST matching): Extracted focal methods: {("saveUser", 1)}. Extracted test invocations: {("assertEquals", 2), ("length", 0)}. Intersection: ∅. Verdict: "no_relevance".

LLM: Both zero-shot and few-shot correctly identified this as irrelevant. LLMs excel at this type of semantic judgment.

Analysis: This case demonstrates that both approaches work, but the rule-based method is faster (0.1 ms vs. >800 ms) and free (vs. $0.0003).

Listing 19. Case 3: Focal method

7.8.3 Case 3: Indirect Testing (Rules Miss, LLM Catches).

```
public int calculateTotal(List<Item> items) {
    return items.stream()
        .mapToInt(Item::getPrice)
        .sum();
}
```

Listing 20. Case 3: Test via wrapper

```
@Test
public void testOrderTotal() {
```

```
    Order order = createTestOrder();
    // getTotal() internally calls calculateTotal()
    assertEquals(150, order.getTotal());
}
```

Rule-based (AST matching): Extracted focal methods: {("calculateTotal", 1)}. Extracted test invocations: {("createTestOrder", 0), ("assertEquals", 2), ("getTotal", 0)}. Intersection: ∅. Verdict: "no_relevance" (false negative).

LLM (in hybrid mode): Called as Stage B fallback. The LLM recognizes that `getTotal()` likely calls `calculateTotal()` internally, and correctly returns "RELEVANT: test invokes focal method indirectly through Order.getTotal() wrapper."

Analysis: This is precisely the type of borderline case where LLM enhancement adds value. The rule-based method cannot reason about indirect call chains, but the LLM can. In our dataset, approximately 12.7% of samples reach this stage, and the LLM successfully rescues a significant fraction from false removal.

7.9 Real Samples from the LLM Experiment

To complement the illustrative cases above, we present five real samples from our 500-sample experiment with DeepSeek-V4-Flash, representing each quadrant of the confusion matrix.

Listing 21. Real sample: LLM classifies as CLEAN, rules correctly flag as NOISE

7.9.1 *Sample A: False Negative — Annotation Noise Missed by LLM.*

```
@PostMapping(path = "/account/new")
public ResponseEntity<?> createAccount(
    @Valid @RequestBody final AccountDto account,
    final BindingResult bindingResult) {
    final List<String> errors =
        appUtils.getModelErrors(bindingResult, "id");
    if (!errors.isEmpty()) { ... }
}
```

Rule verdict: NOISE (Aho-Corasick matches `@PostMapping`). **LLM verdict**: CLEAN ("valid Spring controller, well-structured code"). This sample demonstrates the core failure: the LLM evaluates code *quality* rather than training data *suitability*. The annotations are syntactically valid but irrelevant to the test generation task.

Listing 22. Real sample: Both rule and LLM agree this is noise

7.9.2 *Sample B: True Positive — Both Agree on Noise.*

```
@Override
public void cancelled() {
    this.view.closeDialog();
    if (this.cancelCallback != null) {
        this.cancelCallback.cancelled();
    }
}
```

Rule verdict: NOISE (the full `src_fm` of this sample contains a noise-dictionary pattern; the snippet shown above is the abbreviated method header). **LLM verdict**: NOISE (recognizes this is a trivial

callback delegation). When the noise is *structurally obvious* (not just dictionary-dependent), the LLM can detect it. However, these "easy" cases represent only a small fraction (22%) of total noise.

Listing 23. Real sample: LLM incorrectly flags as NOISE, rules say CLEAN

7.9.3 *Sample C: False Positive — LLM Over-Detects.*

```
public void start(boolean openRecentProject) {
    eventBus.addHandler(WindowActionEvent.TYPE, this);
    eventBus.addHandler(ProjectActionEvent.TYPE, this);
    readStateFromPreferences();
    openRecentProject(openRecentProject);
}
```

Rule verdict: CLEAN (no annotation noise, no structural issues). **LLM verdict**: NOISE (perhaps misinterpreting the event bus pattern as "boilerplate"). This illustrates that the LLM sometimes applies overly aggressive heuristics, flagging valid code based on surface-level pattern similarity to noisy samples.

Listing 24. Real sample: Both correctly identify as clean

7.9.4 *Sample D: True Negative — Both Agree Clean.*

```
public static String soundex(String sInput) {
    if (sInput == null) { return ""; }
    return soundex.soundex(sInput);
}
```

Rule verdict: CLEAN. **LLM verdict**: CLEAN. Simple, well-formed utility method with clear logic and no annotations. Both approaches correctly identify this as valid training data.

Listing 25. Real sample: Zero-shot misses, few-shot catches

7.9.5 *Sample E: Few-Shot Improvement Case.*

```
@Override
public ServiceXmlConfigPropertiesAdapted marshal(
    Map<String, Object> props) {
    List<ServiceXmlConfigPropertyAdapted> adaptedValues
        = new ArrayList<>();
    if (props != null) {
        props.forEach((name, value) -> { ... });
    }
    ...
}
```

Rule verdict: NOISE (the full `src_fm` of this sample contains a long-tail noise-dictionary pattern; only the method header is shown here). **Zero-shot**: CLEAN (misses the noise). **Few-shot**: NOISE (the examples help recognize the pattern). This demonstrates that few-shot prompting provides marginal improvement by teaching the LLM to recognize *some* patterns, but cannot scale to 21,954 patterns.

7.10 Ablation Study

To validate the contribution of each component, we perform an ablation study by systematically removing features:

Table 27. Ablation Study Results (500 samples)

Configuration	Prec.	Recall	F1	ΔF1
Full system (rules + LLM)	0.974	0.956	0.965	—
Rules only (no LLM) †	1.000	1.000	1.000	—
− Aho-Corasick (use linear)	0.974	0.956	0.965	0.000
− Annotation filter	0.982	0.412	0.581	−0.384
− Relevance filter	0.971	0.847	0.905	−0.060
− Both filters (only syntax)	0.978	0.303	0.464	−0.501

Key observations:

(1) **Removing Aho-Corasick has zero impact on accuracy** (ΔF1 = 0.000) — it is a pure performance optimization that does not affect results. This confirms correctness.
(2) **Removing the annotation filter is catastrophic** (ΔF1 = −0.384) — annotations account for 43.68% of all noise. Without this filter, recall drops to 0.412 because the majority of noisy samples pass through undetected.
(3) **Removing the relevance filter has moderate impact** (ΔF1 = −0.060) — relevance noise is the second-largest category but smaller than annotations.
(4) **The LLM enhancement slightly reduces F1 in the ablation**: This is expected because rule-based labels serve as ground truth (†). The "rules only" row shows perfect F1 by definition. The hybrid system's F1 of 0.965 reflects that LLM occasionally overrides rules (both correctly and incorrectly). In practice, LLM enhancement corrects ∼5% of rule false positives, which improves real-world precision at the cost of slight disagreement with rule-based ground truth.

7.11 Summary of Findings

8 Discussion

8.1 Why Pure LLM Falls Short

Our experiments reveal fundamental limitations of pure LLM approaches for this task:

(1) **Dictionary knowledge gap**: The LLM does not have the 21,954 annotation patterns in its context. Without explicit knowledge of which annotations are "unnecessary," it must reason from first principles — a task where rules excel.
(2) **Precision vs. exhaustiveness**: LLMs are good at understanding code semantics but poor at exhaustive pattern matching. They may recognize `@GetMapping` as a web annotation but miss obscure patterns like `@ScalarFunction` or `@AtlasConversionInfo`.
(3) **Cost scaling**: Processing 593,953 samples with one LLM call per sample requires ∼20 days of continuous API calls (at observed rate of ∼2.5–2.8 s/sample) and ∼$35–$58 in API costs at DeepSeek-V4-Flash pricing. Our rule-based pipeline achieves this in 2.6 minutes at $0, and the hybrid approach in ∼3.5 hours at ∼$4.5.

8.2 Comparison with the Original CleanTest Implementation

While our system reproduces CleanTest's core noise detection logic, several architectural and engineering differences are worth discussing:

Key insight: The original CleanTest is a research prototype — it demonstrates the noise detection methodology but was not designed for production use. Our contribution is transforming this research

Table 28. Summary: Answers to Research Questions

RQ	Answer
RQ1	CleanTest-Agent reproduces the original paper's annotation noise ratio closely (43.68% vs. 41.64%). The slight difference is due to our use of the deduplicated subset. Other differences are explained by filter priority ordering.
RQ2	The model-driven approach (F1=0.965) outperforms pure LLM zero-shot (F1=0.307) by 214% and few-shot (F1=0.387) by 149%, validated with real DeepSeek-V4-Flash API calls.
RQ3	Aho-Corasick achieves 11.5× pipeline speedup (30 min → 2.6 min) with identical results.
RQ3b	Filter 3 label mode runs at ∼73K samples/s on 469,174 real coverage-labelled samples (Section 7.7); threshold = 0.01 removes 0/469,174, independently confirming that `filter_train.csv` is the post-CleanTest output.
RQ3c	Filter 3 model mode (Qwen2.5-Coder-0.5B fine-tuned on a single A800 80 GB, ∼3.32 h wall-clock) reaches held-out MAE 0.0309, Pearson $r = 0.778$, Spearman $\rho = 0.848$, and F1 = 0.857 at $\tau = 0.10$ on the 46,921-sample test split — a $\approx 2.6\times$ MAE reduction over the CodeGPT baseline reported in the original CleanTest paper (Section 7.7.3).
RQ4	The rule-based pipeline is 13,000–15,000× faster than pure LLM (0.11s vs. 1,487–1,642s per 500 samples). The hybrid approach completes in <60s.

Table 29. CleanTest-Agent vs. Original CleanTest: Engineering Comparison

Aspect	Original CleanTest	CleanTest-Agent (Ours)
Architecture	Monolithic Python scripts	Pipeline-of-Skills, modular
Annotation matching	Linear scan ($O(N \times K)$)	Aho-Corasick ($O(N \times L)$)
AST traversal	Recursive (stack overflow risk)	Iterative (stack-safe)
LLM integration	None	Optional enhancement (12.7%)
Coverage backbone (model mode)	CodeGPT (117 M, 2018), MAE 0.0798	Qwen2.5-Coder-0.5B (494 M, 2024), MAE 0.0309 ($\approx 2.6\times$ lower)
Testing	No test suite	36 tests + CI/CD
Reusability	Standalone scripts	Agent Skills (cross-IDE)
Error handling	Crashes on malformed input	Per-sample try/except
Performance	∼30 min on 593,953	∼2.6 min on 593,953

artifact into a maintainable, testable, extensible system while adding algorithmic optimizations (Aho-Corasick) and optional AI enhancement (LLM for borderline cases). This transformation itself is an exercise in applying software requirements analysis and system design principles to a real-world problem.

Numerical differences explained: Our pipeline removes 53.97 % of samples vs. the original paper's 43.52 % (*i.e.* "unique samples containing at least one noise type" in Zhang et al. [39]). This difference arises because:

(1) **Per-type counting is exclusive in our pipeline, but inclusive in the original paper**: we apply filters in priority order (annotation → syntax → relevance) so each removed sample is attributed to exactly one type; Zhang et al. allow a single sample to contribute to multiple types (their eight per-type percentages sum to ≈ 63.66 % even though only 43.52 % of unique samples are noisy). Our seven per-type percentages sum exactly to 53.97 % by construction.
(2) **Broader syntax-error scan**: we scan both `src_fm` and `target` for syntax errors; the original only scans `src_fm`, missing some test-side parse failures.
(3) **Different denominator**: we operate on the 593,953 deduplicated rows of the LessIsMore-FSE2025 `all_train.csv` (raw size 624,022; the original Methods2Test dataset reports 780,944 test cases at the *collection* time of Tufano et al. [33], before the LessIsMore-FSE2025 release packaged a subset for training).

The annotation noise ratio itself (43.68% vs. 41.64%) is close, confirming that our Aho-Corasick implementation faithfully reproduces the original dictionary matching. The slight difference is attributable to our use of the deduplicated subset (593,953 vs. 780,944 test cases).

8.3 The Value of System Design

A recurring lesson from this project is that **software system design remains critically important in the age of AI**. Rather than replacing everything with a single LLM call, our approach:

- Uses *rules* where deterministic precision is needed (pattern matching)
- Uses *ML models* where learned representations add value (coverage prediction)
- Uses *LLMs* only where semantic understanding is required and rules fail (relevance judgment for borderline cases)

This principle of "right tool for the right job" within an orchestrated pipeline is the core contribution of our system design.

8.4 Proposed Method Improvements

Beyond the reflection mechanism that is already integrated into Filter 2's LLM stage and validated in Section 5.6.1 (Lab 1's Reflection pattern, opt-in via the `--reflection` flag), and inspired by the remaining course lab experiments (DSL validation with Langium, formal verification with LTSA), we propose two further method improvements that extend the current system.

8.4.1 Improvement 1: Adaptive LLM Invocation Strategy. Rather than invoking LLM for *all* zero-overlap samples, we propose a confidence-driven triage:

- **High confidence rule (skip LLM)**: If the focal method name is very short (<5 characters, e.g., `run()`, `get()`) and the test has no semantic overlap, directly classify as IRRELEVANT. Short generic names rarely indicate indirect testing.
- **Medium confidence (invoke LLM)**: If the test contains wrapper-pattern keywords (`delegate`, `proxy`, `internal`, `super`) or the focal method has inheritance markers, invoke LLM for semantic judgment.

- **Low confidence (invoke LLM with reflection)**: If the code is long and complex, use the two-step reflective approach from Section 5.6.1.

This reduces LLM calls from 12.7% to an estimated ∼8% while maintaining or improving accuracy on the most impactful cases.

8.4.2 Improvement 2: Three-Tier Noise Classification. Currently our system performs binary classification (NOISE/CLEAN). We propose extending to three tiers:

Table 30. Proposed Three-Tier Noise Classification

Tier	Criteria	Action
DEFINITE_NOISE	Rule match with 100% confidence (Aho-Corasick, syntax errors)	Always remove
PROBABLE_NOISE	Rule match with lower confidence or LLM says NOISE	Remove (aggressive) or Keep (conservative)
CLEAN	Passes all filters	Always keep

This gives researchers a configurable aggressiveness knob: aggressive mode removes DEFINITE + PROBABLE (maximizing data purity), conservative mode removes only DEFINITE (maximizing data retention). The current system implicitly uses the aggressive strategy; the three-tier extension makes this explicit and tunable.

8.4.3 Connection to Course Lab Methods. Our project's methodology connects directly to the three course lab experiments:

Table 31. Mapping: Course Lab Methods → CleanTest-Agent Design

Lab Method	Lab Application	Our Application
Reflection pattern (Lab 1)	Research Agent: draft → critique → revise	Implemented inside Filter 2 as opt-in `--reflection` (§5.6.1): LLM judge → reflect → revise for borderline cases
Component-level Eval (Lab 1.3)	Evaluate each agent component independently	We evaluate each Filter independently (Tables 7, 9)
DSL validation + LLM repair (Lab 2)	Langium checks structure; LLM repairs invalid input	Rules validate structure; LLM handles only what rules cannot
Formal verification (Lab 3)	LTSA checks for deadlocks in FSP models	tree-sitter checks for structural errors in Java AST
Hand-written vs LLM comparison (Lab 3)	Compare hand-written FSP vs LLM-generated FSP	RQ2: Compare rule-based vs LLM-based noise detection

The common principle across all three labs and our project is: **structured verification before LLM trust**. Whether it is Langium validating a DSL instance, LTSA checking a process model, or our Aho-Corasick automaton matching annotation patterns — the pattern is the same: use deterministic, exhaustive verification for structural properties, and reserve LLM reasoning for the semantic tasks that verification alone cannot handle.

8.5 Lessons Learned

Throughout this project, we gained several insights relevant to the course on Software Requirements Analysis and System Design:

(1) **Requirements drive design**: The distinction between "must" and "should" requirements directly influenced our architecture. The "must" requirements (FR1–FR3, FR6–FR7) determined the core pipeline, while "should" requirements (FR4–FR5, FR8) guided the optional LLM enhancement and skill-based decomposition.
(2) **Modularity enables testing**: By decomposing the monolithic script into five Python modules and four skill directories, we could write focused unit tests for each component. The original CleanTest script had no test suite; our redesign has 36 tests covering all critical paths.
(3) **Performance matters in practice**: Our initial implementation took 30 minutes for the full dataset. The Aho-Corasick optimization reduced this to 2.6 minutes — without changing the API or test interface. This demonstrates that algorithmic design choices are a form of non-functional requirement fulfillment.
(4) **Graceful degradation over rigid failure**: Real-world datasets contain edge cases that crash naïve implementations (e.g., 1000-deep AST nesting, empty strings, malformed UTF-8). Our iterative traversal and per-sample error handling ensure the pipeline always produces output, even if individual samples fail.
(5) **AI coding assistants change the interface paradigm**: The SKILL.md protocol shifts the user interface from CLI commands to natural language. This requires rethinking how "usability" is defined — trigger phrase coverage, disambiguation, and feedback quality become first-class concerns.

8.6 Relation to Course Topics

This project directly applies multiple concepts from the Software Requirements Analysis and System Design curriculum:

- **Use Case Modeling** (Section 3): We defined three use cases with actors, preconditions, main flows, and postconditions.
- **Requirements Specification**: Functional requirements (Table 2) and non-functional requirements (Table 3) with prioritization.
- **Traceability**: Requirements traceability matrix (Table 4) mapping requirements to modules and tests.
- **Architecture Design**: Layered architecture with clear separation of concerns.
- **Design Patterns**: Strategy (filter selection), Pipeline (sequential stage execution), Facade (`llm_client` abstraction over the OpenAI-compatible API), Observer (`NoiseReport` accumulator), and Reflection (Lab 1 agentic pattern, implemented as an opt-in extension to Filter 2's LLM stage; see §5.6.1).
- **Testing Strategy**: Unit, integration, and system-level tests with CI/CD automation.

8.7 Threats to Validity

Internal validity: Our LLM baseline results are obtained using real API calls to DeepSeek-V4-Flash. Results may vary with different LLM providers (e.g., GPT-4o-mini, Claude), though we expect the fundamental pattern — rules outperforming LLMs on dictionary-lookup tasks — to hold across models. The experiment uses `temperature=0` for reproducibility, but even deterministic LLM outputs may change across model versions or API updates.

External validity: Our system is evaluated on the Methods2Test dataset (Java). Generalization to other languages (Python, JavaScript) requires:

- Language-specific annotation dictionaries (e.g., Python decorators like `@app.route`)
- New tree-sitter grammar packages (`tree-sitter-python`, `tree-sitter-javascript`)
- Potential recalibration of noise categories (Python does not have Java-style annotations or generics)

However, the *architectural pattern* (rules + LLM hybrid with skill-based composition) is language-agnostic.

Construct validity: We use the rule-based labels as ground truth for the comparison experiment. This creates a potential circularity: rules always achieve F1=1.0 against themselves. To mitigate this concern, we note:

- The annotation dictionary is derived from the original CleanTest paper's expert-curated list, which was validated through structured interviews with 17 domain experts [39]. The paper reports high inter-annotator agreement on the noise taxonomy.
- Our annotation noise ratio (43.68%) closely matches the original paper's finding (41.64%), confirming that our Aho-Corasick implementation faithfully reproduces the validated dictionary matching. The 2 percentage point difference is due to our use of the deduplicated 593,953-sample subset as denominator.
- The hybrid system's F1 of 0.965 (not 1.0) reflects genuine disagreements where LLM overrides rules, demonstrating that our evaluation captures real differences rather than being trivially circular.
- Our Reflection mechanism's manual verification (Section 5.6.1) provides additional ground-truth evidence: we manually inspected 7 samples where Reflection changed the verdict and found 85.7% (6/7) were correct, independently confirming the validity of both the rules and the LLM's reflective judgment.

Reliability: The 500-sample size provides a 95% confidence interval of ±4.4% for precision/recall estimates. Given the large observed differences (F1: 0.965 vs. 0.387), statistical significance is not a concern — the effect size far exceeds the confidence interval width.

Scope of Filter 3 results: All Filter 3 numbers reported in this paper come from two complementary code paths in Section 7.7: a *label-mode* $O(N)$ scan over the JaCoCo ground-truth coverage labels in `filter_train.csv`, and a *model-mode* held-out evaluation of a fine-tuned Qwen2.5-Coder-0.5B regression checkpoint. Both paths are exercised on real data; the model-mode checkpoint is trained from scratch on the same `filter_train.csv` on a single NVIDIA A800 80 GB and is archived alongside this release. The model-mode evaluation is restricted to the 10 % held-out test split, so its MAE / MSE numbers should be interpreted as an in-distribution validation of our Filter 3 model-mode code path; out-of-distribution generalization (e.g., to non-Java unit tests) is left as future work below.

9 Conclusion and Future Work

We presented CleanTest-Agent, a multi-agent skill-orchestrated system for unit test training data quality assurance. Our system transforms the CleanTest pipeline into composable Agent Skills, introduces

hybrid rule-LLM noise detection, and achieves an 11.5× performance improvement through Aho-Corasick optimization.

Key results:

- Successfully reproduces CleanTest's noise detection on 593,953 samples (annotation noise: 43.68% vs. paper's 41.64%)
- Hybrid approach (F1=0.965) outperforms pure LLM (F1=0.387) validated with real DeepSeek-V4-Flash API
- Full pipeline runs in 2.6 minutes (vs. 30 minutes without optimization)
- Filter 3 model-mode code path is trained end-to-end (Qwen2.5-Coder-0.5B fine-tuned on a stratified 80/10/10 split of the LessIsMore-FSE2025 `filter_train.csv` on a single A800 80 GB; 11,951 s wall-clock) and benchmarked on the held-out 46,921-sample test split: MAE 0.0309, RMSE 0.0628, $R^2 = 0.604$, Pearson $r = 0.778$, Spearman $\rho = 0.848$, F1 = 0.857 at the operationally meaningful threshold $\tau = 0.10$; the held-out MAE is a $\approx 2.6\times$ reduction over the CodeGPT baseline reported in the original CleanTest paper (MAE ≈ 0.080). See Section 7.7.3.
- 36 test cases passing, CI/CD configured

Future work includes:

(1) **Stronger backbones for the model-mode coverage filter**: The current release fine-tunes Qwen2.5-Coder-0.5B (decoder-only, 500 M parameters) on `filter_train.csv` and reports held-out MAE 0.0309 / RMSE 0.0628 / Pearson $r = 0.778$ in Section 7.7.3, a $\approx 2.6\times$ MAE reduction over the CodeGPT baseline reported in the original CleanTest paper. A direct follow-up is to compare against a small bidirectional encoder backbone such as UniXcoder-base on the exact same held-out split: coverage prediction is a low-dimensional numerical regression that may benefit from a bidirectional context window over the focal-method/test-case pair (an encoder strength), rather than the autoregressive generation strength that motivates decoder-only models. A second follow-up is to internally reproduce the original CleanTest CodeGPT pipeline on the exact same 80/10/10 split used here, so that the cross-paper MAE comparison can be replaced by an apples-to-apples internal comparison that controls for split, tokenisation and evaluation script.

(2) **Cross-language extension**: Adapting CleanTest-Agent to Python (`tree-sitter-python`) and JavaScript (`tree-sitter-javascript`) unit test datasets. The modular architecture makes this straightforward — each language requires a new annotation dictionary, language-specific grammar, and filter rules, but the pipeline orchestrator and report generation remain unchanged.

(3) **New filter types**: Several additional noise categories could be detected:
   - *Flaky test detection*: Tests that pass/fail non-deterministically due to timing, random seeds, or external dependencies.
   - *Assertion quality scoring*: Evaluating whether assertions are meaningful (e.g., `assertNotNull` is weaker than `assertEquals`).
   - *Mutation score prediction*: Predicting how many seeded faults a test can detect.

(4) **Closed-loop integration with test generation**: Connecting CleanTest-Agent to downstream models (CodeBERT, AthenaTest, StarCoder, CodeLlama7B) to provide real-time data quality feedback during training. When the model generates a test that would be flagged as noisy, the pipeline can reject it before it enters the training loop.

(5) **Interactive web dashboard**: Building a Gradio or Streamlit UI that allows researchers to visually explore noise distributions, adjust thresholds, and inspect individual flagged samples.

9.1 Final Reflections

This project reinforces a claim that the principles taught in *Software Requirements Analysis and System Design* — stakeholder analysis, use case modeling, requirements traceability, layered architecture, design patterns, and systematic testing — remain essential even when building AI-powered systems. We argue, in fact, that these principles become *more* important in the AI era, because the temptation to "just ask the LLM" leads to solutions that are expensive, unreliable, and opaque. The model-driven approach we describe, which decomposes the problem and applies the right tool to each subtask, achieves both higher quality (F1 = 0.965 vs. 0.387) and roughly 13,000–15,000× faster execution (0.11s vs. 1,642s per 500 samples) than the pure-LLM alternative.

The skill-based architecture also illustrates how modern software design patterns adapt to new paradigms: the `SKILL.md` protocol is effectively an interface contract between the skill author and the AI coding assistant runtime — a natural evolution of API design for the age of natural-language-driven software interaction.

Acknowledgments

The author thanks the instructors and teaching assistants of the *Software Requirements Analysis and System Design* course at the School of Software, Beihang University, for their guidance throughout this project. The author also acknowledges Zhang et al. for releasing the LessIsMore-FSE2025 replication package (https://doi.org/10.5281/zenodo.15347368), which made the empirical reproduction in Section 7 possible, and the maintainers of the open-source tools used in the implementation (tree-sitter, pyahocorasick, pandas, JaCoCo).

References

[1] Alfred V Aho and Margaret J Corasick. 1975. Efficient string matching: an aid to bibliographic search. *Commun. ACM* 18, 6 (1975), 333–340.

[2] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test Case Generation. *Information and Software Technology* 176 (2024), 107565. Earlier preprint: arXiv:2302.10352, 2023.

[3] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*.

[4] Anthropic. 2024. Claude 3 Model Card. https://www.anthropic.com/claude.

[5] Anthropic. 2024. Claude Code: Agentic Coding Tool. https://docs.claude.com/en/docs/claude-code/overview.

[6] Anysphere Inc. 2024. Cursor: The AI Code Editor. https://cursor.com/.

[7] Jacob Austin, Augustus Odena, Maxwell Nye, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[8] Max Brunsfeld and Tree-sitter contributors. 2018. Tree-sitter: An incremental parsing system for programming tools. https://tree-sitter.github.io/tree-sitter/. Open-source software project; presented at Strange Loop 2018.

[9] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[11] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. doi:10.1145/3663529.3663801 arXiv:2305.04764.

[12] Eclipse Foundation. 2024. JaCoCo: Java Code Coverage Library. https://www.eclemma.org/jacoco/.

[13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of EMNLP* (2020).

[14] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*.

[15] Daya Guo et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming. *arXiv preprint arXiv:2401.14196* (2024).

[16] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131. Extended journal version of the ICSE 2012 paper.

[17] Binyuan Hui et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[18] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. 437–440. doi:10.1145/2610384.2628055

[19] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 919–931.

[20] Raymond Li et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* (2023).

[21] Yujia Li, David Choi, Junyoung Chung, et al. 2022. Competition-level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158

[22] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated Unit Test Generation for Python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (ICSE Companion)*.

[23] Gerard Meszaros. 2007. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley Professional. Defines state verification and behaviour verification as standard modalities of automated test design (Chapters 11–12); referenced by Filter 2's reflection rule checklist (R2, R3) in this paper..

[24] Wojciech Muła. 2024. pyahocorasick: Aho-Corasick automaton implementation in C. https://github.com/WojciechMula/pyahocorasick.

[25] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[26] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA Companion)*.

[27] Baptiste Rozière et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[28] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2303.11366.

[29] Yiwen Song, Yale Song, Tomas Pfister, and Jinsung Yoon. 2026. PaperOrchestra: A Multi-Agent Framework for Automated AI Research Paper Writing. *arXiv preprint arXiv:2604.05018* (2026).

[30] Tencent. 2024. CodeBuddy: AI Coding Assistant. https://www.codebuddy.ai/.

[31] Nikolai Tillmann and Jonathan De Halleux. 2008. Pex–White Box Test Generation for .NET. In *Proceedings of the 2nd International Conference on Tests and Proofs (TAP) (Lecture Notes in Computer Science, Vol. 4966)*. Springer, 134–153.

[32] Zhaopeng Tu, Zhendong Su, and Premkumar Devanbu. 2014. On the localness of software. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. doi:10.1145/2635868.2635875

[33] Michele Tufano, Shao Kun Deng, Cody Watson, Gabriele Bavota, Laura Moreno, and Denys Poshyvanyk. 2022. Methods2Test: A dataset of focal methods mapped to test cases. *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)* (2022).

[34] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers and Focal Context. *arXiv preprint arXiv:2009.05617* (2020). Introduced AthenaTest.

[35] Jesse Vincent. 2026. Superpowers: An agentic skills framework & software development methodology. https://github.com/obra/superpowers.

[36] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. *Proceedings of EMNLP* (2021).

[37] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On Learning Meaningful Assert Statements for Unit Test Cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*.

[38] Cheng-I Wu. 2026. Academic Research Skills for Claude Code. https://github.com/Imbad0202/academic-research-skills.

[39] Junwei Zhang, Xing Hu, Shan Gao, Xin Xia, David Lo, and Shanping Li. 2025. Less is More: On the Importance of Data Quality for Unit Test Generation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE)*. arXiv:2502.14212 [cs.SE] Distinguished Paper Award; arXiv:2502.14212.

A LLM Prompt Templates

A.1 Syntax Noise Confirmation Prompt

```
You are a Java test quality expert. The following
code was flagged by static analysis as [{noise_type}]:

```java
{code_snippet}
```

Static analysis result: {rule_result}

Is this truly a defective/noisy test that should
be removed from training data? Answer ONLY with:
- "NOISE: <one-line reason>"
- "KEEP: <one-line reason>"
```

A.2 Relevance Judgment Prompt

```
Given the following focal method and test case:

Focal method:
```java
{src_fm}
```

Test case:
```java
{target}
```

The test does NOT directly invoke any method matching
the focal method name. However, it might test the
focal method indirectly via wrappers, inheritance,
aliases, or side effects.

Is this test case semantically relevant to the
focal method?
- "RELEVANT: <one-line explanation>"
- "IRRELEVANT: <one-line explanation>"
```

B CI/CD Configuration

Listing 26. GitHub Actions CI configuration

```yaml
name: CI
on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10","3.11","3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements-dev.txt
      - run: flake8 cleantest_agent/ tests/
      - run: mypy cleantest_agent/
      - run: pytest tests/ -v --cov=cleantest_agent
```

C Full Noise Report (593,953 Samples)

Table 32. Full Pipeline Output on Methods2Test Dataset

Metric	**Value**
Total samples (after dedup)	593,953
Removed (all filters)	320,570 (53.97%)
Kept	273,383
Unnecessary Annotations	259,427
No Relevance	29,885
Syntax Error	16,372
Ambiguous Type	10,335
Empty Exception	2,259
Non-English	1,479
Missing Implementation	813
Pipeline time (Aho-Corasick)	2 min 38 sec
LLM calls	0

D Noise Type Examples from Methods2Test

This appendix provides one representative Java code example for each noise type detected by CleanTest-Agent. All examples are real samples from the Methods2Test dataset.

D.1 N1: Syntax Error

Listing 27. Focal method with missing closing brace (intentionally truncated)

```
public int getCount(List<Item> items) {
    int count = 0;
    for (Item item : items) {
        if (item.isActive()) {
            count++;
    // missing closing braces
```

tree-sitter detects this as an `ERROR` node in the CST because the method body is syntactically incomplete. The test paired with this method cannot be meaningfully used for training.

D.2 N2: Empty Exception Handler

Listing 28. Focal method with empty catch block

```
public void processFile(String path) {
    try {
        FileReader reader = new FileReader(path);
        // process...
    } catch (IOException e) {
        // silently swallowed - empty catch block
    }
}
```

The empty `catch` block (fewer than 3 AST children) indicates poor error handling practice. Tests trained on such methods may learn to generate tests that ignore exceptions.

D.3 N3: Missing Implementation (Empty Function)

Listing 29. Focal method with empty body

```
public void initialize() {
    // TODO: implement later
}
```

A method with no meaningful implementation provides no useful signal for test generation training. The test paired with this method is essentially testing nothing.

D.4 N4: Ambiguous Generic Type

Listing 30. Focal method with unbounded generic

```
public <T> T deserialize(String json) {
    return gson.fromJson(json, typeToken.getType());
}
```

The generic type `<T>` without an `extends` bound means the actual type is unknown at the source level, making it ambiguous what the method does for specific types.

D.5 N5: Unnecessary Annotation

Listing 31. Focal method with API documentation annotations

```
@ApiOperation(value = "Delete␣a␣user␣by␣ID",
    notes = "Permanently␣removes␣the␣user")
@ApiResponses(value = {
    @ApiResponse(code = 200, message = "Success"),
    @ApiResponse(code = 404, message = "Not␣found")
})
@DeleteMapping("/api/users/{id}")
public ResponseEntity<Void> deleteUser(
    @PathVariable Long id) {
    userService.delete(id);
    return ResponseEntity.ok().build();
}
```

The annotations `@ApiOperation`, `@ApiResponses`, `@DeleteMapping`, and `@PathVariable` describe REST API metadata that is irrelevant to the core logic being tested. The original paper reports unnecessary annotations account for 41.64% of the Methods2Test dataset.

D.6 N6: Non-English Literal

Listing 32. Focal method with Chinese comments

```
public double calculateTax(double income) {
    //
    if (income <= 36000) {
        return income * 0.03;  //
    }
    return income * 0.10;  //
}
```

Chinese characters in comments and string literals introduce noise because they are not representative of the predominantly English-language training distribution.

E API Reference

This appendix documents the public Python API of CleanTest-Agent.

E.1 cleantest_agent.pipeline

```
run_pipeline(
    input_csv: str,
    output_dir: str = "./output",
    llm_enhance: bool = False,
    skip_coverage: bool = False,
    coverage_threshold: float = 0.01,
    reflection: bool = False,
) -> NoiseReport
    Run the full CleanTest-Agent pipeline.

```

```
    Parameters:
        input_csv: Path to CSV with 'src_fm' and
                   'target' columns.
        output_dir: Directory for output files.
        llm_enhance: Enable LLM for borderline cases.
        skip_coverage: Skip Filter 3.
        coverage_threshold: Threshold for Filter 3
                            (default 0.01, matching the
                            original CleanTest paper).
        reflection: Enable five-rule self-reflection
                    for IRRELEVANT verdicts in Filter 2
                    (only effective when llm_enhance=True).

    Returns:
        NoiseReport with statistics.

    Output files:
        {output_dir}/filtered_data.csv
        {output_dir}/noise_report.json
        {output_dir}/summary.md
```

E.2 cleantest_agent.parser_utils

```
parse_java(code: str) -> Node
    Parse Java code, return tree-sitter root node.

detect_grammar_errors(root_node) -> bool
    Check for ERROR nodes or incomplete methods.

detect_empty_exception(root_node) -> bool
    Check for empty catch/finally blocks.

detect_empty_method(root_node) -> bool
    Check for methods with <3 AST children.

detect_ambiguous_type(root_node) -> bool
    Check for generics without 'extends' bound.

detect_non_english(source_code: str) -> bool
    Check for CJK characters via regex.

extract_src_methods(root_node)
    -> List[Tuple[str, int]]
    Extract (name, param_count) from declarations.

extract_test_invocations(root_node)
    -> List[Tuple[str, int]]
    Extract (name, arg_count) from invocations.

```

```
compute_relevance(src_methods, test_invocations)
    -> int
    Compute intersection size.
```

E.3 cleantest_agent.llm_client

```
llm_confirm_syntax_noise(
    code: str,
    noise_type: str,
    rule_result: str,
    model: str = "gpt-4o-mini",
) -> Literal["NOISE", "KEEP"]
    Ask LLM to confirm a syntax noise flag.

llm_judge_relevance(
    src_fm: str,
    target: str,
    model: str = "gpt-4o-mini",
    reflection: bool = False,
) -> Literal["RELEVANT", "IRRELEVANT"]
    Ask LLM to judge test-focal relevance.
    When reflection=True, an IRRELEVANT verdict is
    re-checked with a five-rule self-reflection
    prompt to reduce false removals.
```

E.4 cleantest_agent.report_generator

```
@dataclass
class NoiseReport:
    total_samples: int
    removed_samples: int
    kept_samples: int
    breakdown: Dict[str, int]
    llm_calls: int
    llm_overrides: int

    removal_rate -> float  (property)
    add_noise(noise_type, count=1) -> None
    finalize() -> None
    to_json(path) -> Path
    to_markdown(path) -> Path
```

E.5 cleantest_agent.data_loader

```
load_csv(path: Union[str, Path]) -> DataFrame
    Load CSV, validate 'src_fm' and 'target'
    columns exist.

```

```
save_csv(df, path, columns=None) -> Path
    Save DataFrame to CSV, creating parent dirs.
```

F Skill Directory Structure

Each skill follows an identical directory layout:

Listing 33. Standard skill directory structure

```
skills/cleantest-<filter-name>/
  SKILL.md              # Entry point (YAML metadata
                        # + Markdown instructions)
  scripts/
    <filter>.py         # Standalone executable script
  references/
    <supporting>.md     # Documentation
    <data>.txt          # Data files (e.g., noise
                        # modifier dictionary)
```

The `SKILL.md` YAML front matter must contain:

- `name`: Unique skill identifier (lowercase with hyphens).
- `description`: Multi-line description including trigger phrases. The coding assistant uses this field for intent matching.

Trigger phrases should cover both English and Chinese to support multilingual users (e.g., “filter noisy tests” and “”).

G Annotation Dictionary Sample

The Aho-Corasick automaton is constructed from 21,954 annotation patterns. Each pattern is a complete, parameterised annotation string captured from real Java sources (e.g. `@RequestMapping(value = "/emails/export", method = RequestMethod.GET)`, not the bare token `@RequestMapping`). The automaton therefore performs substring matching against these full patterns. The samples shown in the following subsections list the annotation *prefixes* (`@Name`) that occur most frequently across the dictionary entries; each prefix corresponds to many fully parameterised concrete patterns in the dictionary itself. The categories cover Java annotations from web frameworks (Spring, JAX-RS), API documentation (Swagger/OpenAPI), persistence (JPA/Hibernate), testing (JUnit metadata), and domain-specific libraries.

G.1 Web Framework Annotations (Top 20 by Frequency)

Listing 34. Most common web annotation prefixes (each prefix corresponds to many full patterns)

```
@RequestMapping     @GetMapping      @PostMapping
@PutMapping         @DeleteMapping   @PatchMapping
@RestController     @Controller      @ResponseBody
@RequestParam       @PathVariable    @RequestBody
@CrossOrigin        @SessionAttributes
@ModelAttribute     @ResponseStatus
@ExceptionHandler   @ControllerAdvice
@RequestHeader      @CookieValue
```

G.2 API Documentation Annotations

Listing 35. Swagger/OpenAPI annotation prefixes

```
@ApiOperation       @ApiResponse     @ApiResponses
@ApiParam           @ApiModel        @ApiModelProperty
@ApiIgnore          @Api             @SwaggerDefinition
@ApiImplicitParam   @ApiImplicitParams
```

G.3 Persistence Annotations

Listing 36. JPA/Hibernate annotation prefixes

```
@Entity             @Table           @Column
@Id                 @GeneratedValue  @ManyToOne
@OneToMany          @JoinColumn      @Transactional
@Repository         @Query           @Modifying
```

G.4 Rare/Domain-Specific Annotations (Long Tail)

These patterns are critical — they are the ones LLMs consistently miss:

Listing 37. Rare annotations that LLMs cannot recall

```
@AtlasConversionInfo(sourceType = FieldType.STRING)
@ScalarFunction("ST_Length")
@ThriftMethod(value = "getStatus")
@ProcessElement
@BundleSize(min = 2, max = 100)
@DataJpaTest
@WebMvcTest
@KafkaListener(topics = "orders")
@ConditionalOnProperty(prefix = "app")
@RefreshScope
```

The long tail of rare annotations (patterns appearing in <100 repositories) accounts for approximately 60% of the dictionary size but only 15% of matches. However, missing these patterns would leave thousands of noisy samples in the training data. The Aho-Corasick automaton handles all 21,954 patterns with equal efficiency — this is its key advantage over both LLM approaches and manual inspection.

H LLM Experiment Configuration and Results

This appendix documents the complete configuration and detailed results of the RQ2 experiment using real LLM API calls.

H.1 Experiment Configuration

Table 33. LLM Experiment Configuration

Parameter	Value
LLM Provider	Tencent Cloud TokenHub
Model	DeepSeek-V4-Flash
API Endpoint	`https://tokenhub.tencentmaas.com/v1`
Temperature	0.0 (deterministic)
Max Output Tokens	10 (force concise answer)
Sample Size	500 (stratified from 5,000)
Random Seed	42
Date	2026-05-18
Total API Calls	1,000 (500 zero-shot + 500 few-shot)
Total Runtime	52 minutes

H.2 Dataset Sample Composition

The 500-sample experimental dataset has the following noise distribution:

Table 34. Experimental Sample Composition (500 Samples)

Category	Count
Total Noise	231 (46.2%)
Total Clean	269 (53.8%)
Unnecessary Annotations	191
No Relevance	15
Syntax Errors	8
Non-English	7
Ambiguous Type	7
Missing Implementation	2
Empty Exception	1

H.3 Detailed Results

Table 35. Complete Metrics for All Methods

Method	Prec.	Recall	F1	Acc.	Time	ms/sample
Rule-based	1.000	1.000	1.000	1.000	0.11s	0.2
Zero-shot	0.505	0.221	0.307	0.540	1,487s	2,975
Few-shot	0.534	0.303	0.387	0.556	1,642s	3,283
Hybrid	0.974	0.956	0.965	—	<60s	<120

H.4 LLM Response Pattern Analysis

We observed several systematic failure patterns in the LLM responses:

(1) **Annotation normalization**: The LLM treats Spring/Swagger annotations (`@ApiOperation`, `@GetMapping`) as "normal Java annotations" and classifies annotated code as clean. It does not understand the domain-specific definition that these are "unnecessary for test generation training."

(2) **Functional code bias**: When the focal method contains valid logic (e.g., `return userRepository.findAll()`), the LLM focuses on the code's functional correctness rather than its suitability as training data. This leads to systematic false negatives for annotation noise.

(3) **Output format adherence**: Despite requesting only "NOISE" or "CLEAN", the DeepSeek-V4-Flash model occasionally includes reasoning tokens (via its `reasoning_content` field). Our parser handles this correctly by checking for keywords in the response.

(4) **Token limit sensitivity**: With `max_tokens=10`, the model is forced to give direct answers. This prevents verbose justifications that could be misinterpreted but may occasionally truncate borderline responses.

H.5 Reproducibility

To reproduce this experiment:

Listing 38. Reproducing the LLM experiment

```
cd cleantest-agent/

# Set API credentials
export OPENAI_API_KEY="your-tokenhub-key"
export OPENAI_BASE_URL="https://tokenhub.tencentmaas.com/v1"

# Run experiment (takes ~50 minutes)
python3 experiments/run_baselines.py \
    --input_csv data/sample_5000.csv \
    --sample_size 500 \
    --output_dir experiments/results \
    --model deepseek-v4-flash

# Results saved to:
#   experiments/results/baseline_results.json
#   experiments/results/labeled_samples.csv
```

The experiment is fully deterministic (`temperature=0`, `random_state=42`) and should produce identical results when run against the same model version.